Beyond Static Dispatching Heuristics: A Comparative Evaluation of Deep Reinforcement Learning under Disruptions in Open-Pit Mining

Jean-Luc Mupasa Kalunga[1,*] Olfa Ferchichi[2]

[1]Faculty of Computer Science, Université Don Bosco de Lubumbashi (UDBL), Lubumbashi, DRC

[2]RIADI Laboratory, ENSI, University of Manouba, Tunis, Tunisia

16mk293@esisalama.org; olfaferchichi@yahoo.fr

Abstract

Truck–shovel haulage can account for up to 60% of operating costs in an open-pit mine and remains highly sensitive to disruptions. Reinforcement learning studies of truck dispatching usually evaluate policies in a single operating regime, against simple heuristics and from a single training run, so their actual value and their behaviour under disruption remain poorly understood. This article compares ten dispatching policies in a stochastic discrete-event simulator: six heuristics, including a rule based on predicted waiting time and a policy based on a linear program, two tabular reinforcement learning agents, Q-Learning and State–Action–Reward–State–Action, and two deep reinforcement learning agents, Deep Q-Network and Proximal Policy Optimization. Because decisions do not all last the same time, the problem is formulated as a semi-Markov decision process with per-minute discounting. The agents are tuned under a rule fixed in advance, on seeds distinct from the evaluation seeds, and each is trained with five seeds. Three regimes are studied: nominal conditions, a fleet enlarged by half, and frequent truck breakdowns; agents trained under nominal conditions are also evaluated without retraining in the two disrupted regimes. Each combination of policy and regime is evaluated over 30 paired replications, with non-parametric tests corrected for multiple comparisons and confidence intervals over training seeds. The rule based on predicted waiting time is the most productive in all three regimes. Proximal Policy Optimization follows closely, 0.5 to 0.9% below, on par with the linear-programming policy; it uses 1.4 to 2.8% less fuel per tonne and keeps its performance without retraining in the disrupted regimes and in two unseen regimes. Deep Q-Network collapses onto the waiting action in four of its five nominal models, whatever the waiting penalty, and the tabular agents degrade as exploration decreases. No agent therefore outperforms the best heuristic: the value of learning lies here in the trade-off with fuel and in stability outside the training regime, and it should be measured against queue-aware rules rather than static ones.

Keywords: Deep reinforcement learning; Open-pit mining; Truck dispatching; Fleet management; Operational resilience; Stochastic simulation; Proximal policy optimization; Deep Q-network.

1 Introduction

Open-pit mining is a cornerstone of the global raw-material supply chain, where the economic viability of operations is inseparable from the efficiency of large-scale logistics systems. It is therefore essential to explore recent technological advances to improve the performance and resilience of these logistics systems.

1.1 Industrial context

Open-pit mines are very large industrial systems in which the truck–shovel cycle forms the main logistics backbone. This process, which covers excavation, haulage and dumping of material, is

not merely an operational task but a decisive financial factor, accounting for up to 60% of the total operating costs of a mine site [2]. In modern mining, the pressure to optimise these flows is twofold: maximising throughput to meet global demand for raw materials, while minimising the energy consumption and greenhouse-gas emissions of diesel haulage fleets.

The efficiency of these systems is, however, constantly challenged by high operational variability. In real mining environments, conditions are far from deterministic. Equipment breakdowns, payload fluctuations, random travel times caused by road degradation and sudden traffic congestion together create a highly unstable environment. Managing this complexity requires real-time decision making that balances immediate productivity and fleet resilience.

1.2 Key concepts

This study relies on a few concepts used throughout the article. Reinforcement learning (RL) is a branch of artificial intelligence in which an autonomous agent learns a behaviour by trial and error, through its interactions with a dynamic environment, so as to maximise a cumulative reward signal. Deep reinforcement learning (DRL) extends RL by using neural networks to approximate value functions or policies, which makes it possible to handle high-dimensional state spaces. Truck dispatching is the real-time operational process of assigning trucks to loading and dumping points in order to keep the fleet synchronised and productive; in mines, it is implemented by fleet management systems (FMS).

Four learning algorithms are studied. Q-Learning and State–Action–Reward–State–Action (SARSA) are tabular methods that store one value for each state–action pair. Deep Q-Network (DQN) replaces this table with a neural network. Proximal Policy Optimization (PPO) learns a policy directly with an actor–critic method. The six reference heuristics are referred to below as First-In-First-Out (FIFO), Fixed Assignment, Nearest Shovel, Shortest Path, Minimum Truck Waiting Time (MTWT) and LP, a policy based on a linear program; their exact definitions are given in Section 5.7. Finally, the decision problem is formulated as a Markov decision process (MDP), more precisely as a semi-Markov decision process (SMDP), since decisions are separated by variable durations (Section 5.4).

1.3 Objective and organisation of the article

The objective of this work is to determine in which operating regimes a learned dispatching policy is preferable to an industrial rule, and to explain the observed differences from the behaviour of the policies. The contribution is evaluative: no new algorithm is proposed, and the contribution lies in a controlled comparison protocol applied to ten policies and three regimes.

The rest of the article is organised as follows. Section 2 (Related work) reviews the truck-dispatching literature along three paradigms. Section 3 (Positioning and problem statement) summarises this literature in a positioning table and derives the research gap, the problem studied, the research questions and the hypotheses. Section 4 (Contributions and methodological framework) presents the contributions and the methodological framework. Section 5 (Proposed approach: model, simulator and agents) describes the modelled system, the simulator, the formulation of the decision problem and the agents studied. Section 6 (Experimental design) details the experimental protocol and Section 7 (Results) the results. Section 8 (Discussion) discusses these results and their limitations, and Section 9 (Conclusion and future work) concludes the article.

2 Related work

The optimisation of truck–shovel systems has been at the heart of mining engineering and operations research for more than six decades. As Newman et al. point out [26], these systems have evolved from manual planning to complex automated fleet management systems (FMS). This section reviews the literature along three main paradigms: static heuristics, mathematical programming and the emerging field of reinforcement learning.

2.1 Static and rule-based truck dispatching

Rule-based dispatching remains the dominant paradigm in operating mines, because it is transparent, computationally cheap and easy to integrate into commercial FMS software. Early reviews of computerised dispatching strategies [24] and the later overview of solution strategies by Alarie and Gamache [5] identify a small set of recurring decision rules that still shape industrial practice today.

Four rules are particularly relevant to this study. The First-In-First-Out (FIFO) rule gives priority to the loading unit that has been waiting for a truck the longest; it balances shovel workload but ignores the spatial layout of the pit. Fixed Assignment allocates each truck to a predetermined shovel for the whole shift; it stabilises the match factor and remains a widespread industrial reference, but cannot reallocate capacity in response to queue imbalances. Nearest Shovel sends the truck to the closest loading unit, which minimises empty travel distance at the price of a purely local optimisation. Shortest Path generalises this idea by choosing the destination that minimises an estimated cycle cost on the haul-road graph. The exact definitions used in this study are given in Table 6.

The practical performance of these rules has been assessed both in the field and in simulation. Subtil et al. [39] report a practical implementation of dispatching in an operating open-pit mine, while Mohtasham and Mirzaei-Nasirabad [23] evaluate a commercial dispatching package at the Sungun copper mine and stress the sensitivity of rule-based logic to operating conditions. Discrete-event simulation has been the standard instrument for this kind of evaluation [52, 14, 49, 1]. It consistently shows that greedy rules degrade sharply once the fleet size exceeds the loading capacity of the pit, because queues concentrate on a subset of shovels. The heuristics retained here represent different logics: FIFO takes the state of the queues into account, Fixed Assignment imposes a static and balanced allocation, while Nearest Shovel and Shortest Path rely on spatial proximity without considering queues. Two stronger references, of the kind used by commercial systems, complete the set: a rule combining queues and travel times, Minimum Truck Waiting Time [49], and a policy guided by an upstream linear-optimisation layer [37, 36].

2.2 Mathematical optimisation and metaheuristics

To overcome the limits of simple rules, researchers turned to mathematical programming. Soumis et al. [37] proposed a multi-stage approach combining linear programming (LP) for production planning with assignment models for real-time dispatching. This modular structure remains the backbone of many modern FMS. Ahangaran et al. [4] extended this approach by developing models dedicated to truck fleets of heterogeneous capacities, using network flows to balance production.

However, the deterministic nature of mixed-integer linear programming (MILP) often conflicts with the stochastic reality of mining. Ta et al. [43] attempted to bridge this gap by incorporating shovel idle probabilities into linear models, using queueing theory to obtain better production estimates. Despite these advances, MILP models frequently suffer from the curse of dimensionality when applied to large mines operating dozens of trucks [36, 46].

To handle this complexity, metaheuristics have been widely applied to truck dispatching and routing. Souza et al. [38] showed that a hybrid heuristic combining GRASP and VNS could obtain near-optimal solutions in short computing times. Liu and Chai [21] used a genetic algorithm to minimise haulage energy consumption while accounting for time-varying road resistance. Ant colony optimisation algorithms, which model the probabilistic route selection of social insects through pheromone update mechanisms, have been applied to multi-vehicle assignment problems in surface mining, as their stochastic selection naturally adapts to competing demands on shared road segments [45]. Simulated annealing, which accepts sub-optimal moves with a temperature-controlled probability in order to escape local optima, has likewise been used for scheduling in open-pit environments [28]. Although powerful, metaheuristics are often formulated to optimise a given system state or planning horizon. When major disruptions change the underlying operating conditions, the optimisation must be repeated or re-run, which may limit responsiveness when decisions must be updated on short time scales in dynamic mining environments.

2.3 Reinforcement learning approaches

The shift to reinforcement learning represents a change of paradigm: instead of solving a new optimisation problem at each decision epoch, an agent learns a policy that maps system states to actions [40]. Early applications of RL in logistics, such as those of Nazari et al. [25] for the vehicle routing problem (VRP) and of Zhang et al. [50] for job-shop scheduling, showed that DRL could outperform classical heuristics by learning from thousands of simulated interactions.

Within the reinforcement learning family, methods differ substantially in representational power and learning paradigm. Tabular methods, such as Q-Learning and SARSA [40], build value functions over tables of discrete state–action pairs, which makes them transparent and computationally light, but their expressiveness degrades sharply as the observation space grows with fleet size. DQN agents address this problem by approximating the Q-function with a neural network and stabilising training through experience replay and a frozen target network, which allows them to generalise to high-dimensional continuous inputs. PPO [35] belongs to the actor–critic family: it optimises the policy directly through a surrogate objective whose clipping constraint prevents catastrophic parameter updates. These four algorithms, from tabular to deep and from value-based to policy-gradient, form a natural comparative spectrum and constitute the RL component of the testbed designed in this study.

In mining, this transition is more recent. Hazrathosseini and Afrapoli [17] provide a critical review, stressing that, although RL holds considerable potential for intelligent FMS, most studies remain at the proof-of-concept stage. De Carvalho and Dimitrakopoulos [11] applied an actor–critic framework to production scheduling, showing that RL can handle fleet dynamics while updating production targets.

Recent testbeds, such as Mining-Gym [6], have begun to provide standardised environments for testing these algorithms. Zhao et al. [51] proposed a hierarchical multi-agent reinforcement learning (MARL) approach to sustainable dispatching with an emphasis on energy efficiency, while Pei et al. [29] and Rosales and Diaz [32] explored learned dispatching and interpretable dispatching for autonomous mine haulage. The use of RL-based decision support for hierarchical industrial scheduling extends beyond mining: Szatmári et al. [42] review similar multi-level architectures in chemical process control, which suggests that the design patterns emerging in mining-dispatching research are relevant across domains. A common limitation of these studies, however, is their focus on nominal conditions. Most RL agents are trained and tested in stable environments; few studies explicitly evaluate how a DQN or PPO agent behaves under frequent equipment breakdowns or unexpected congestion peaks.

3 Positioning and problem statement

Table 1 positions the study with respect to the families of work reviewed in Section 2; the following subsections derive the research gap, the problem studied, the research questions and the hypotheses.

Table 1: Positioning of the proposed study with respect to representative truck-dispatching approaches.

Approach	Formal optimisation	Learning	Disruption tests	Baselines	Replication protocol	Focus of the evaluation
Reviews of dispatching and fleet management [26, 24, 5, 17, 2]	n/a	n/a	No	n/a	n/a	Overview of models and algorithms
Rule-based dispatching, field and simulation [39, 23, 52, 14, 1]	No	No	No	Current practice	Limited	Field or discrete-event validation
Comparison of rules by fleet size [49]	Genetic algorithm	No	Congestion	4 to 6 rules	Limited	Comparison of allocation strategies
Mathematical programming [37, 4, 43, 36, 46]	Yes (LP / MILP)	No	No	0 to 2	Limited	Optimality under stable conditions
Metaheuristics and simulation-optimisation [38, 21, 45, 28]	Yes (heuristic search)	No	Scenario-based	MILP or 1 to 2 rules	Limited	Near-optimal planning solutions
Learning-based dispatching, other domains [25, 50, 42]	No	Deep and hierarchical RL	No	Classical heuristics	Yes	Routing and scheduling performance
Reinforcement learning in mining [11, 6, 51, 29, 32]	No	Actor–critic, MARL, interpretable RL	No	0 to 3	Limited	Proof-of-concept dispatching
Proposed study	**No (heuristics as baselines)**	**Tabular and deep RL (Q-Learning, SARSA, DQN, PPO)**	**Congestion and breakdowns**	**6 heuristics, incl. MTWT and LP**	**30 paired replications, 5 training seeds**	**Operational resilience across three regimes**

3.1 Research gaps

The literature reviewed and Table 1 reveal four complementary gaps.

First, the three paradigms reviewed are rarely compared with one another. Rule-based studies compare dispatching rules with each other [5, 23, 49], mathematical programming compares an optimiser with a rule or with a mixed-integer formulation [37, 36], and learning-based studies compare an agent with a small number of baseline rules [11, 51]. When cross-family comparisons exist, the algorithms are evaluated on different environments, rewards or fleet configurations, which prevents attributing an observed difference to the dispatching logic itself.

Second, the operating regime is usually treated as a fixed context rather than as an experimental factor. Reviews of RL-based fleet management observe that most contributions remain at the proof-of-concept stage and are evaluated in stable environments [17], and studies that do consider disruptions usually consider only one type, most often congestion [49]. A reported ranking is therefore conditional on a single operating point.

Third, the strength of the baselines is often insufficient. Learning-based studies report improvements over two or three baseline rules [51], and sometimes over none [11]. When the comparison rule is weak, an improvement establishes little about the practical value of the learned policy.

Fourth, the inferential protocol is rarely documented. The number of replications, the control of random seeds and corrections for multiple comparisons are rarely reported in the mining-dispatching literature [23, 1, 28], whereas the literature on the evaluation of reinforcement learning agents has shown that these choices change the conclusions [18, 3].

3.2 Research problem

The central challenge of dispatching is the dynamic, real-time assignment of trucks to loading and dumping points so as to optimise an objective combining throughput, idle time and energy consumption. Fleet management systems have been deployed in mines for several decades [2, 26], and the decision rules behind them are well documented [24, 5]. When a disruption occurs, such as a breakdown or congestion, it can propagate through the haulage network as queues and equipment under-utilisation; discrete-event studies thus show that queueing increases with fleet size as soon as loading capacity is approached [49, 52].

The practical question is therefore not only whether adaptive dispatching improves productivity under stable conditions, but whether the choice of a policy remains the right one when the operating regime changes. The problem is studied at two levels.

> **Problem P1 – Real-time dispatching decision.** Given the current state of the mine, assign the truck that has just become available to a shovel–dump pair, or make it wait, so as to maximise the tonnage delivered over a shift while limiting queueing and fuel consumption.

> **Problem P2 – Choice of a policy by regime.** Given a set of dispatching policies and a set of operating regimes, determine which policy performs best in each regime, whether the ranking obtained under nominal conditions holds under disruption, and whether a policy learned under nominal conditions keeps its performance when deployed without retraining in another regime.

Problem P1 is formalised in Section 5.4. Problem P2 is addressed by the experimental protocol of Section 6.

3.3 Research questions and hypotheses

The empirical evaluation answers four research questions:

RQ1 Which policy achieves the highest productivity in each regime, and does this ranking hold from one regime to another?

RQ2 How do productivity and queueing time evolve jointly with the policy and the regime?

RQ3 Does a policy learned under nominal conditions keep its performance when deployed without retraining in a disrupted regime, compared with a policy trained in that regime?

RQ4 Does the ranking by specific fuel consumption follow the productivity ranking?

RQ1 and RQ2 are addressed in Sections 7.1 to 7.4, RQ3 in Section 7.5 and RQ4 in Section 8.4. The statistical significance of each comparison is examined in Section 7.6.

Three falsifiable hypotheses, formulated before the analysis of the results, guide the study:

- **H1:** under nominal conditions, deep reinforcement learning policies achieve higher productivity than the best heuristic.
- **H2:** no policy dominates in all three regimes: the ranking depends on the regime, which translates into a significant interaction between policy and regime.
- **H3:** high productivity does not necessarily come with low queueing time, so that several policies form a trade-off between these two criteria.

3.4 Claims and scope limitations

The contribution of this article is evaluative rather than algorithmic: no new reinforcement learning algorithm is proposed. Nor does it rest on the existence of a mining simulation environment, since comparable environments already exist, such as Mining-Gym [6]. It lies in the combination of several elements rarely brought together: ten policies from different families, including two strong baselines, in the same environment, several operating regimes treated as an experimental factor, an evaluation of learned policies without retraining (also carried out by Pei et al. [29] for autonomous haulage), a paired-replication protocol with corrected tests, and an analysis of policy behaviour that explains the observed differences.

Three scope limitations follow from this positioning and are discussed in Section 8.5. The evaluation is carried out in simulation, so the reported magnitudes describe relative performance in the proposed simulated environment rather than site-level predictions. The agents are single controllers that decide for all trucks, so multi-agent formulations [51] fall outside the compared space. Finally, the three regimes represent stylised disruptions, designed to be interpretable and reproducible.

4 Contributions and methodological framework

4.1 Contributions

This work makes four contributions.

- **C1. A multi-regime evaluation protocol.** A stochastic simulator compatible with the Gymnasium interface is combined with a measurement protocol that treats the operating regime as an experimental factor, evaluates each policy over 30 paired replications and applies paired tests corrected for multiple comparisons.
- **C2. A comparison of ten policies from different families.** Six heuristics, including a rule based on predicted waiting time (MTWT) and a policy based on a linear program (LP), two tabular learning agents and two deep learning agents are evaluated in the same environment, with the same actions, the same seeds and the same indicators.
- **C3. An analysis of policy behaviour.** The allocation of cycles among shovels and the use of the waiting action, measured over all replications, explain the differences in productivity and queueing between policies.
- **C4. Evidence on regime dependence.** The ranking of policies is compared across regimes, the interaction between policy and regime is tested, and an evaluation without retraining shows to what extent a policy learned under nominal conditions keeps its performance under disruption.

4.2 Methodological framework

Figure 1 summarises the steps of the study. (1) The haulage system is modelled and implemented in a stochastic simulator (Sections 5.1 and 5.3). (2) The dispatching decision is formulated as a semi-Markov decision process with a multi-objective reward (Sections 5.4 and 5.5). (3) The four learning agents are trained and the six heuristics are implemented (Sections 5.6 and 5.7). (4) All policies are evaluated in three regimes over 30 paired replications, and the nominal agents are also evaluated without retraining in the disrupted regimes (Section 6). (5) The results are analysed with paired statistical tests and an analysis of policy behaviour (Section 7).

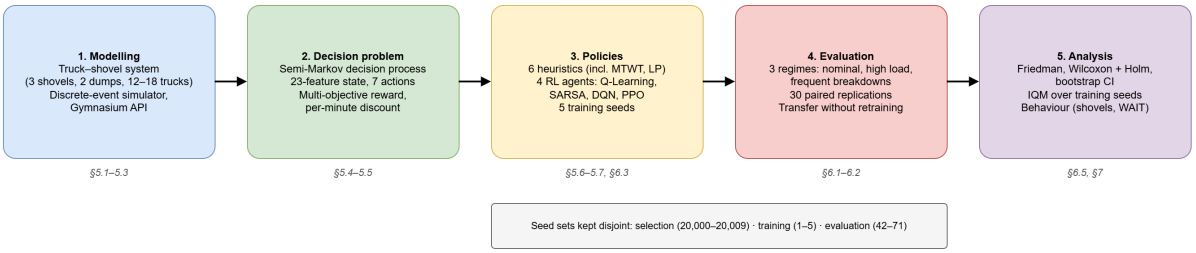


Figure 1: Methodological framework of the study.

5 Proposed approach: model, simulator and agents

This section describes the modelled system, the simulator, the formulation of the decision problem, the reward function, the learning agents and the reference heuristics. All values given are those of the evaluated implementation.

5.1 Truck–shovel haulage system

The mine is modelled as a closed-loop haulage system made up of three physical entities:

1. **Loading units (shovels):** fixed resources characterised by their loading time. Each shovel serves one truck at a time; a truck arriving during a loading waits until the shovel is free.
2. **Haulage units (trucks):** a homogeneous fleet of 140-tonne trucks, characterised by their empty and loaded speeds and their fuel consumption.
3. **Dumping units (dumps):** the destinations of the material. Like shovels, they serve one truck at a time.

The system is subject to truck breakdowns. At the end of each cycle, a truck breaks down with a fixed probability (2% or 10% depending on the regime) and remains immobilised for a duration drawn uniformly between 10 and 30 minutes. Shovels and dumps do not break down in this study.

5.2 Stochastic road network

The mine infrastructure is represented by a directed graph $G = (V, E)$, where V contains the parking area, an intersection, the shovels and the dumps, and E the haul roads connecting them. Each edge carries a distance, a grade and a surface condition. The travel time $T(e)$ of an edge e follows a log-normal distribution centred on the mean time $\bar{T}(e)$:

$$T(e) \sim \text{Lognormal}(\mu_e, \sigma^2), \qquad \mu_e = \ln \bar{T}(e) - \frac{\sigma^2}{2}, \qquad \sigma = 0.12, \tag{1}$$

where $\bar{T}(e)$ depends on the distance, the base speed (26 km/h loaded, 32 km/h empty), a penalty of 3% per grade point and the road condition. At each pass over an edge, the road condition degrades by 0.02 with probability 5% (floor of 0.6); roads are restored at the start of each shift. All parameters are gathered in Table 2.

5.3 Simulation environment

The simulator follows the Gymnasium interface [44], which ensures compatibility with Stable-Baselines3 [31]. The software architecture is shown in Fig. 2.

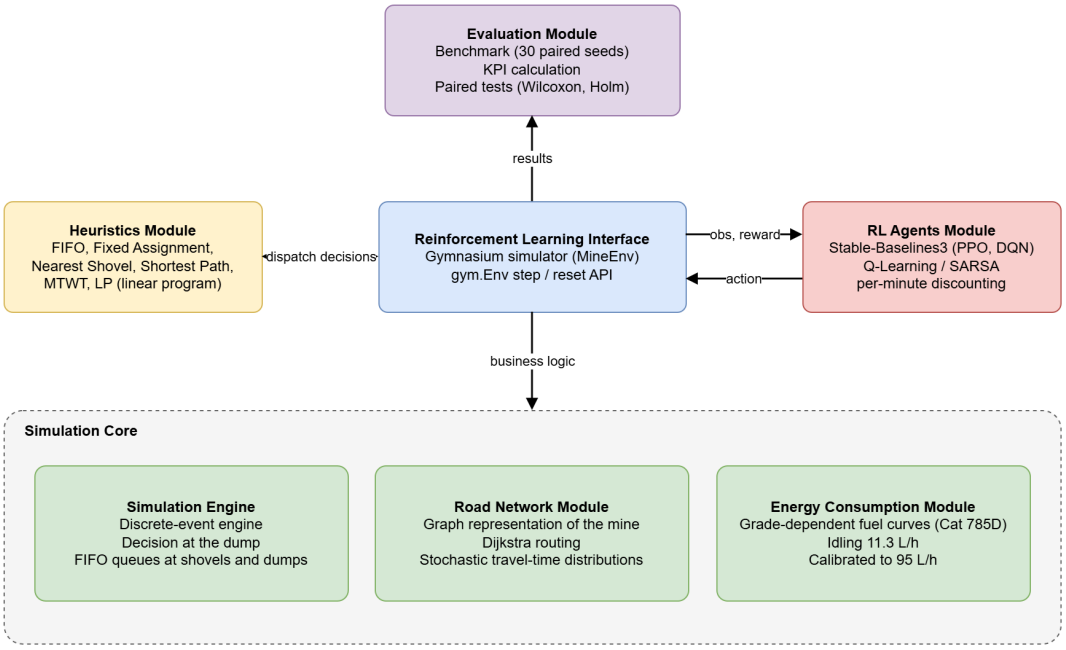


Figure 2: Modular software architecture of the evaluation framework.

The architecture comprises the following modules:

- **Discrete-event simulation engine:** a decision is requested whenever a truck becomes available, at the parking area for its first cycle, then at the dump where it has just unloaded. A cycle goes from the truck's location to the chosen shovel, then to the chosen dump, and ends at the dump after unloading and a possible breakdown. The decision only starts the empty trip. Queueing and loading are simulated when the truck arrives at the shovel, queueing and dumping when it arrives at the dump, and these events are processed in the chronological order of the whole fleet. Shovels and dumps therefore serve trucks in their order of arrival (first come, first served). The clock advances from event to event until the next available truck.
- **Road network module:** graph representation of the mine, route computation with Dijkstra's algorithm and sampling of travel times.
- **Fuel consumption module:** the reference truck is the Cat 785D (target payload 133 t, rated payload 145 t [9, 10]). When idling (queues, loading, dumping, breakdowns, voluntary waiting), it consumes 11.3 L/h, a value measured on an operating 785D [20]. When travelling, consumption per kilometre follows the curves measured in the same study as a function of grade, loaded and empty, increased according to road condition. Applied as is, these curves give 211 L/h per truck for Fixed Assignment, above the published maximum for this truck (135.6 L/h [9]): all roads in the simulator climb, return trips included, and it represents neither descent nor coasting. They are therefore multiplied by a factor of 0.447, chosen on the selection seeds so that Fixed Assignment consumes 95 L/h, the middle of the published "medium" range [9]. The average level thus comes from the manufacturer, and the split between idling, load and grade comes from measurements. Over-consumption on degraded roads remains an uncalibrated assumption (Section 8.5).
- **Randomness:** the seed of an episode generates an independent random stream per process (travel, loading and dumping, road degradation) and one breakdown stream per truck. Two policies played with the same seed thus share the same random events (common random numbers): in particular, the k-th cycle of a given truck suffers the same breakdown, of the same duration, whatever the policy.
- **Learning interface:** the Gymnasium `reset`–`step` API, through which the agent observes the

state, chooses an action and receives a reward.

- **Indicator and benchmark modules:** computation of the indicators of Section 6.4 for all policies, with the same seeds.

Table 2: Simulator parameters.

Item	Value
Topology	7 nodes: parking, intersection, 3 shovels, 2 dumps; directed edges, direct shovel–dump links
Distances	parking–intersection 0.8 km; intersection–shovels 1.5 / 1.8 / 2.2 km; intersection–dumps 2.8 / 3.2 km; shovel–dump 0.8× (sum of the two previous distances)
Grades	access to shovels 4 / 5 / 6%; access to dumps 6 / 8%; return grade reduced by 2 to 3 points (minimum 1%)
Speeds	26 km/h loaded, 32 km/h empty; factor $\max(0.55\,;\,1-0.03\times\text{grade})\times$ road condition; minimum 5 km/h
Travel time	log-normal around the mean time, $\sigma = 0.12$; minimum 0.2 min
Loading	log-normal, mean 2.0 min, standard deviation 0.3 min (capacity of one shovel $\approx$ 30 loads/h, i.e. $\approx$ 4,200 t/h)
Dumping	log-normal, mean 1.0 min, standard deviation 0.2 min
Road degradation	probability 5% per pass of losing 0.02 of condition (floor 0.6); restored at each shift
Breakdowns	trucks only; probability per cycle 2% (nominal, high load) or 10% (frequent breakdowns); repair $\mathcal{U}(10, 30)$ min
Trucks, shift	140 t per truck; 480-min shift; a cycle is counted only if it ends before the end of the shift
Fuel	travel: $0.447 \times f(x) \times (1 + 0.5 \times (1 - \text{condition}))$ L/km, where, for a grade x (fraction), $f(x) = 7.638 + 80.825\,x + 603.87\,x^2 - 803.8\,x^3$ loaded and $f(x) = 4.562 + 6.809\,x + 727.7\,x^2 - 1661\,x^3$ empty [20]; idling 11.3 L/h [20]
Randomness	one independent random stream per process and one breakdown stream per truck, generated from the episode seed (common random numbers across policies)

5.4 Formulation as a semi-Markov decision process

Problem P1 is formulated as a Markov decision process [30, 40] defined by the tuple (S, A, P, R, γ). Since decisions are triggered by events (a truck becoming available) and separated by variable durations, it is more precisely a semi-Markov decision process [7, 41]. Figure 3 illustrates the interaction loop; Table 4 summarises the components.

5.4.1 State space

The state S_t is a vector of 23 components, normalised to $[0, 1]$, which describes the system as seen by the deciding truck (Table 3). Its size is $3S + 2D + SD + 4$ and does not depend on the number of trucks: 23 components with 12 as with 18 trucks. An agent trained in one regime can therefore be evaluated in another without retraining (Section 6.2).

The block of predicted waiting times per route gives, for each of the six shovel–dump pairs, the waiting time the deciding truck would face if it took that route: the predicted queue on its arrival at the shovel, then the predicted queue on its arrival at the dump after an average loading. The predictions account for trucks already en route. This is exactly the information used by the Minimum Truck Waiting Time rule (MTWT, Section 5.7). This block was added during agent tuning (Section 6.3): without it, the agent had to learn on its own the non-linear combination $\max(0, \text{release} - \text{arrival})$ that MTWT computes.

Table 3: Composition of the state vector ($S = 3$ shovels, $D = 2$ dumps).

Block	Size	Content and normalisation
Waiting at shovels	3	time until each shovel is free, trucks en route included, divided by 480 min
Waiting at dumps	2	same quantity for each dump
Trucks per shovel	3	trucks committed to each shovel (en route, queueing or loading), divided by 24
Distances	3	distance from the deciding truck to each shovel, divided by 10 km
Predicted waiting per route	6	predicted waiting time of the deciding truck on each shovel–dump route (queue at the shovel and queue at the dump), divided by 30 min and capped at 1
Position	3	location of the deciding truck (parking or one of the two dumps), one-hot encoding
Deciding truck	2	tonnage delivered since the start of the shift / 3,500 t; fuel consumed / 1,085 L
Time	1	time elapsed in the shift, divided by 480 min
Total	23	independent of the number of trucks

5.4.2 Action space

The action A_t is requested whenever a truck becomes available. Each action encodes a complete cycle assignment: a shovel–dump pair ($\text{Shovel}_j, \text{Dump}_k$), or a waiting action (WAIT). With $S = 3$ shovels and $D = 2$ dumps:

$$|A| = S \times D + 1 = 7, \quad A_t \in \{0, 1, \ldots, 6\}, \tag{2}$$

where action $a \in \{0, \ldots, 5\}$ corresponds to $\text{Shovel}_{\lfloor a/D \rfloor}$ and $\text{Dump}_{a \bmod D}$. Action 6 holds the truck until the next system event (release of a truck, a shovel or a dump), for at most 5 minutes and at least 0.2 minute. No action mask is used: the seven actions are always allowed, for all agents.

5.4.3 Transition function and discounting

The transition function $P(S_{t+1} \mid S_t, A_t)$ is defined implicitly by the simulator: travel times, loading and dumping durations, road degradation and breakdowns. The same action taken in the same state can therefore lead to different successor states.

Duration of decisions. Two successive decisions are separated by a variable duration Δt_t: from 0.2 min after a short WAIT to several minutes, depending on when the next truck becomes available. Over a shift, the durations of the decisions cover exactly the 480 minutes.

Per-minute discounting. A reward obtained after a decision that lasted Δt_t minutes is discounted by $\gamma_{\min}^{\Delta t_t}$, rather than by a fixed factor γ at each decision [7, 41]. The discounted return from decision t is therefore

$$G_t = \sum_{l \geq 0} \Big(\prod_{m=0}^{l-1} \gamma_{\min}^{\Delta t_{t+m}} \Big) R_{t+l} = \sum_{l \geq 0} \gamma_{\min}^{\tau_{t+l} - \tau_t} R_{t+l}, \tag{3}$$

where τ_t is the time of decision t in the shift. The time that counts becomes mine time, not the number of decisions.

No new parameter. The per-minute factor is derived from the γ of each algorithm through $\gamma_{\min} = \gamma^{1/\bar{\Delta}}$, where $\bar{\Delta} = 1.89$ min is the mean interval between two decisions of Fixed Assignment under nominal conditions. A decision of average duration is therefore discounted exactly as with a per-decision factor. With $\gamma = 0.995$, the effective horizon remains on average about 200 decisions, i.e. about 377 minutes under nominal conditions. It does not depend on fleet size, whereas with per-decision discounting it would shrink in minutes when decisions become more frequent, for instance under high load.

Why. With per-decision discounting, a decision lasting 0.2 min and a cycle to the farthest shovel delay what follows equally, although one lasts a hundred times longer than the other. Agent tuning

revealed two measured consequences of this flaw (Section 6.3). Value-based agents drifted towards the farthest shovel, since a long trip does not delay the next production in number of decisions. When a cycle yielded a negative value on average, chaining waits of 0.2 min pushed this cost back by many decisions for only a few minutes. Per-minute discounting fixes the common cause of both flaws without changing the reward.

Implementation. For Q-Learning and SARSA, the target becomes $R_t + \gamma_{\min}^{\Delta t_t} Q(S_{t+1}, \cdot)$. For DQN, the replay buffer stores the factor $\gamma_{\min}^{\Delta t}$ of each transition, which Stable-Baselines3 uses in the target. For PPO, generalised advantage estimation uses a per-step factor:

$$\delta_t = R_t + \gamma_{\min}^{\Delta t_t} V(S_{t+1}) - V(S_t), \qquad \hat{A}_t = \delta_t + \gamma_{\min}^{\Delta t_t} \lambda \hat{A}_{t+1}. \tag{4}$$

Bootstrapping the value at the end of the shift uses the same factor. A test checks that, if all decisions lasted exactly 1.89 min, the four algorithms would learn exactly what they learn with per-decision discounting: same network weights for PPO and DQN, same tables for Q-Learning and SARSA.

Approximations. There are two. First, the reward of a step is counted at the start of the step, without discounting within it, which is customary for a semi-Markov process with per-transition rewards. Second, for PPO, reward normalisation estimates the scale of returns with the per-decision γ. It only rescales the reward and does not enter the target. Finally, per-minute discounting assumes a single simulated environment per training run.

Credit assignment. A single agent decides successively for all trucks. The reward of a step covers the cycles completed during that step, whatever the truck (Section 5.5): the credit of a decision to a particular truck is therefore not explicit.

Table 4: Components of the semi-Markov decision process.

Component	Notation	Description
Environment	$\mathcal{E}$	Discrete-event simulator: 3 shovels, 2 dumps, N trucks, 480-min shift (Table 2).
State	$S_t \in [0,1]^{23}$	Waiting at shovels and dumps, trucks per shovel, distances, predicted waiting per route, position, tonnage and fuel of the deciding truck, shift time (Table 3).
Action	$A_t \in \{0, \ldots, 6\}$	Assignment to one of the 6 shovel–dump pairs or WAIT (index 6).
Transition	$P(S_{t+1} \mid S_t, A_t)$	Stochastic simulator: log-normal travel, loading and dumping, road degradation, breakdowns.
Reward	R_t	Sum of the rewards of the cycles completed during the step, and cost of a WAIT (Eq. (5)).
Discounting	$\gamma_{\min}^{\Delta t_t}$	Per simulated minute; $\gamma_{\min} = \gamma^{1/1.89}$ (Section 5.4.3).

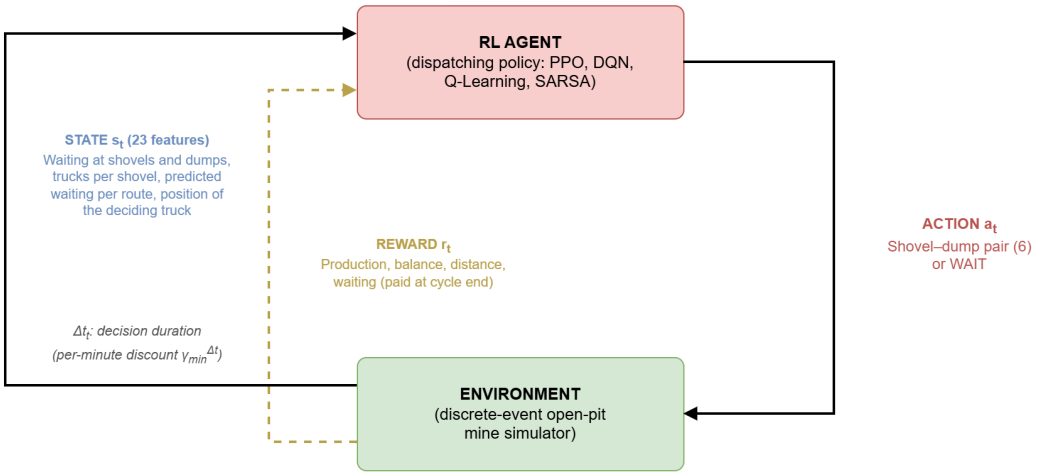


Figure 3: Formulation of the truck-dispatching problem as a decision process.

5.5 Multi-objective reward function

The agent's objective is to maximise the discounted cumulative reward over a shift (Eq. (3)). The reward of a cycle c, decided for truck i, is:

$$r_c = w_1\, r_c^{\text{prod}} - w_2\, p_c^{\text{bal}} - w_3\, c_c^{\text{dist}} - w_4\, p_c^{\text{wait}}, \tag{5}$$

with $w_1 = 1$, $w_2 = 0.1$, $w_3 = 0.05$ and $w_4 = 0.3$, and where:

- $r_c^{\text{prod}} = q_c/Q$ is the production of the cycle, relative to the capacity $Q = 140$ t: it equals 1 if the cycle ends before the end of the shift and 0 otherwise;
- $p_c^{\text{bal}} = \frac{1}{100}\,\text{Var}_j(\omega_j)$ penalises imbalance between shovels, where ω_j is the remaining time, in minutes, before shovel j is free at the time of the decision;
- $c_c^{\text{dist}} = (d(x_i, j) + d(j, k))/5$ is the distance of the cycle in kilometres, divided by 5, where x_i is the location of the truck (parking or dump), j the chosen shovel and k the chosen dump; the empty return is the first trip of the next cycle;
- $p_c^{\text{wait}} = \min\left((W_c^{\text{shovel}} + W_c^{\text{dump}})/30\,;\,1\right)$ penalises the queueing time actually incurred during the cycle, in minutes, divided by 30 and capped at 1.

Timing of payment. The reward of a cycle is known only at its end, since queues depend on the arrivals of the other trucks. It is paid at the step during which the cycle ends. The reward R_t of a step is therefore the sum of the rewards of the cycles completed since the previous decision, whatever the truck, plus the cost of a WAIT if that is the chosen action. At the end of the shift, cycles in progress are completed: the sum of the R_t over a shift is the sum of the rewards of all its cycles. A version in which the reward was estimated and paid at the time of the decision was tried and rejected during tuning (Section 6.3).

Cost of voluntary waiting. For the WAIT action, of duration δ_t, the reward is

$$R_t^{\text{wait}} = -w_4 \min(\delta_t/30\,;\,1) - w_1\,\frac{\delta_t}{23.2}. \tag{6}$$

The second term is the opportunity cost of idleness: the production lost by a stopped truck, in cycles, where 23.2 min is the mean cycle duration of Fixed Assignment under nominal conditions. It is weighted by w_1, since it is expressed in the same unit as production, and therefore adds no weight

to tune. Without it, a wait of 0.2 min would cost only about 0.002, and waiting would be almost free. Policies that never wait, including all heuristics, are unaffected by this term.

Order of magnitude. With these normalisations, the cumulative reward of a shift is close to the number of completed cycles, reduced mainly by the distance cost. For Fixed Assignment under nominal conditions (mean over the 10 selection seeds, Section 6.2), 241.5 delivered cycles give a reward of 218.7, of which −19.3 for distance, −2.1 for balance and −1.4 for waiting.

Unchanged reward. The reward was fixed before the campaign. A variant that extended the opportunity cost to all cycles was tried during tuning and then abandoned (Section 6.3). The flaw it targeted was fixed by per-minute discounting (Section 5.4.3), without changing the reward.

The weights favour production while keeping a waiting penalty. The sensitivity of the agents to w_4 is examined in the ablation study (Section 7.9).

5.6 Learning agents studied

5.6.1 Proximal Policy Optimization

PPO [35] is an actor–critic algorithm that optimises the policy directly. Its objective uses a clipped surrogate function to avoid excessively large policy updates:

$$L^{\mathrm{CLIP}}(\theta) = \hat{\mathbb{E}}_t\Big[\min\left(r_t(\theta)\hat{A}_t,\ \mathrm{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\Big], \qquad r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\mathrm{old}}}(a_t \mid s_t)}, \tag{7}$$

where the advantage $\hat{A}_t$ is estimated with generalised advantage estimation (GAE) [34], using the per-minute factor of Section 5.4.3. The actor and the critic are two separate multilayer perceptrons, each with two hidden layers of 128 neurons and a hyperbolic tangent activation, the Stable-Baselines3 default for PPO; the output layer of the actor defines a distribution over the 7 actions. The reward is normalised during training (Stable-Baselines3 `VecNormalize`; observations are not normalised, since they already lie in $[0, 1]$). A single simulated environment is used.

The hyperparameters are gathered in Table 5. For PPO and DQN, the learning rate and, depending on the algorithm, the entropy coefficient or the exploration duration were chosen by a tuning of the same size for both agents: four configurations each, 200,000 steps, training seed 1, productivity criterion on the selection seeds. The factor $\gamma = 0.995$ and PPO's reward normalisation were retained during agent tuning (Section 6.3). DQN was given the same γ, but not the normalisation: its replay buffer would store rewards normalised with statistics that change during training. For Q-Learning and SARSA, the learning rate and the number of discretisation intervals were chosen by a tuning of the same size (four configurations each, full budget). Their factor $\gamma = 0.99$ was not tuned: the tuning rule kept it outside the grid. The other values follow the Stable-Baselines3 defaults and usual settings [35, 22].

Training budgets are expressed in different units: the tabular methods are trained for 30,000 episodes, each episode comprising about 250 decisions, and the deep agents for 2 million decisions, i.e. about 8,000 shifts. The two budgets are not directly comparable.

Table 5: Hyperparameters of the learning algorithms. Values chosen by tuning are in bold.

Hyperparameter	**PPO**	**DQN**	**Q-Learning**	**SARSA**
Representation	MLP [128, 128]	MLP [128, 128]	table, 6 variables (Section 5.6.3)	
Activation	Tanh[a]	ReLU[a]	n/a	n/a
Discretisation intervals	n/a	n/a	**6**	**6**
Learning rate α	$\mathbf{1 \times 10^{-4}}$	$\mathbf{3 \times 10^{-4}}$	**0.2**	**0.1**
Factor γ (1.89-min decision)	0.995	0.995	0.99	0.99
Discounting	per minute, $\gamma_{\min} = \gamma^{1/1.89}$ (Section 5.4.3)			
Reward normalisation	yes	no	no	no
Batch size	64	64	n/a	n/a
n_{steps} / n_{epochs}	1,024 / 10	n/a	n/a	n/a
GAE λ / clipping ϵ	0.95 / 0.2	n/a	n/a	n/a
Entropy coefficient	**0**	n/a	n/a	n/a
Replay buffer / warm-up	n/a	200,000 / 1,000	n/a	n/a
Target network update	n/a	every 1,000 steps	n/a	n/a
Exploration ε	n/a	$1 \to 0.02$, **10%** of steps	$1 \to 0.01$, over episodes	
Training budget	2,000,000 steps	2,000,000 steps	30,000 episodes	30,000 episodes

[a] Default of the SB3 library: Tanh for on-policy algorithms (PPO); ReLU for off-policy algorithms (DQN).

5.6.2 Deep Q-Network

DQN [22] approximates the action-value function $Q(s, a)$ with a neural network, and stabilises learning with an experience replay buffer and a periodically updated target network. With per-minute discounting, the update targets

$$y = r + \gamma_{\min}^{\Delta t} \max_{a'} Q_{\theta^-}(s', a'), \tag{8}$$

where Δt is the duration of the decision, stored with each transition in the replay buffer. The network has two hidden layers of 128 neurons with a ReLU activation. Exploration follows an ε-greedy strategy whose rate decreases linearly from 1.0 to 0.02 over the first 10% of training steps.

5.6.3 Tabular agents: Q-Learning and SARSA

Q-Learning [47] and SARSA [33] store one value for each state–action pair in a table. The 23-component state being too large for a table, it is reduced to six variables. The first five are the waiting times at the 3 shovels and the 2 dumps, split into 6 equal intervals between 0 and 9.6 min (beyond which the value saturates). The sixth is the index of the shovel nearest to the deciding truck (0 to 2). A variant that replaced it with the route of smallest predicted waiting time (0 to 5), i.e. the information used by MTWT, was tried and not retained (Section 6.3). The discretised space therefore contains $6^5 \times 3 = 23{,}328$ states. When a state is absent from the table, action 0 is played by default; the share of decisions concerned is measured during evaluation. This representation does not depend on the number of trucks.

Q-Learning updates the value of the action from the best next action (off-policy method), whereas SARSA uses the action actually chosen next (on-policy method). With per-minute discounting, the update reads

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma_{\min}^{\Delta t} Q(s', a') - Q(s, a) \right], \tag{9}$$

where a' is the highest-valued action in s' for Q-Learning, and the action chosen in s' for SARSA. Both agents use $\gamma = 0.99$ and an ε-greedy exploration decreasing linearly from 1.0 to 0.01 over 30,000 episodes. Q-Learning uses $\alpha = 0.2$ and SARSA $\alpha = 0.1$, values chosen by tuning (Section 6.3). The comparison between tabular and deep agents should therefore be read with the representation and computing budget retained here.

Algorithms 1, 2 and 3 formalise the learning methods as pseudocode; the line that distinguishes SARSA from Q-Learning is indicated at the end of Algorithm 3. In all three, Δt_t is the duration of decision t, given by the simulator, and $\gamma_{\min} = \gamma^{1/1.89}$.

Algorithm 1 PPO for mine truck dispatching

Input: Mine environment $\mathcal{E}$; actor π_θ and critic V_ϕ (MLP $[128, 128]$, Tanh)
Input: α=10^{-4}, γ=0.995, λ_{GAE}=0.95, ϵ_{clip}=0.2, N=1024, $T_{\max}$=2×10^6
Initialise θ, ϕ randomly; $\gamma_{\min} \leftarrow \gamma^{1/1.89}$
for $k = 1, 2, \ldots, \lfloor T_{\max}/N \rfloor$ **do**
  for $t = 1, \ldots, N$ **do** ▷ Collect rollout
    Observe s_t (Table 3)
    Sample $a_t \sim \pi_\theta(\cdot \mid s_t)$, $a_t \in \{0, \ldots, 6\}$ (shovel–dump pair or WAIT)
    Execute a_t; receive r_t (cycles completed during the step, Eq. (5)), s_{t+1} and the duration Δt_t
    Normalise r_t; store $(s_t,\ a_t,\ r_t,\ \gamma_{\min}^{\Delta t_t},\ \log \pi_\theta(a_t \mid s_t))$
  end for
  Compute advantages: $\hat{A}_t = \delta_t + \gamma_{\min}^{\Delta t_t} \lambda_{\text{GAE}} \hat{A}_{t+1}$, $\delta_t = r_t + \gamma_{\min}^{\Delta t_t} V_\phi(s_{t+1}) - V_\phi(s_t)$
  for each epoch, mini-batches of size 64 **do**
    $r_t(\theta) \leftarrow \pi_\theta(a_t \mid s_t) / \pi_{\theta_k}(a_t \mid s_t)$
    Maximise $L^{\text{CLIP}}(\theta) = \mathbb{E}\big[\min\big(r_t \hat{A}_t,\ \text{clip}(r_t, 1-\epsilon_{\text{clip}}, 1+\epsilon_{\text{clip}})\hat{A}_t\big)\big]$
    Minimise $L^V(\phi) = \mathbb{E}\big[(V_\phi(s_t) - R_t^{\text{target}})^2\big]$
  end for
end for
Output: Trained policy π_θ^*

Algorithm 2 DQN for mine truck dispatching

Input: $\mathcal{E}$; networks Q_θ, Q_{θ^-} (MLP $[128, 128]$, ReLU); replay buffer $\mathcal{B}$ (200,000)
Input: α=3×10^{-4}, γ=0.995, ε: $1.0 \rightarrow 0.02$ (linear over 10% of steps)
Initialise θ randomly; $\theta^- \leftarrow \theta$; $\mathcal{B} \leftarrow \emptyset$; $\gamma_{\min} \leftarrow \gamma^{1/1.89}$
for each step $t = 1, 2, \ldots, T_{\max}$ **do**
  With prob. ε: $a_t \sim \text{Uniform}(\mathcal{A})$; otherwise $a_t \leftarrow \arg\max_a Q_\theta(s_t, a)$
  Execute a_t; observe r_t, s_{t+1}, Δt_t; store $(s_t, a_t, r_t, s_{t+1}, \gamma_{\min}^{\Delta t_t})$ in $\mathcal{B}$
  Sample a mini-batch $\{(s_i, a_i, r_i, s_i', d_i)\}$ from $\mathcal{B}$
  $y_i \leftarrow r_i + d_i \max_{a'} Q_{\theta^-}(s_i', a')$ ▷ Target discounted by the decision duration
  Minimise $\frac{1}{B}\sum_i \big(Q_\theta(s_i, a_i) - y_i\big)^2$
  if $t \bmod 1000 = 0$ **then** $\theta^- \leftarrow \theta$
  end if ▷ Synchronise target network
end for
Output: $\pi(s) = \arg\max_a Q_\theta(s, a)$

Algorithm 3 Q-Learning for mine truck dispatching (tabular reference)

Input: $\mathcal{E}$; α=0.2 (SARSA: 0.1), γ=0.99, ε: 1.0→0.01 (linear over 30,000 episodes)

Initialise $Q(s,a) \leftarrow 0$ for all (s,a) (state reduced to 6 variables, 6 intervals); $\gamma_{\min} \leftarrow \gamma^{1/1.89}$
for episode $= 1, \ldots, 30{,}000$ **do**
    $s_0 \leftarrow \texttt{reset}(\mathcal{E})$
    while the episode is not over **do**
        With prob. ε: $a_t \sim \text{Uniform}(\mathcal{A})$; otherwise $a_t \leftarrow \arg\max_a Q(s_t, a)$
        Execute a_t; observe r_t, s_{t+1}, Δt_t
        $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\big[r_t + \gamma_{\min}^{\Delta t_t} \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)\big]$
        $s_t \leftarrow s_{t+1}$
    end while
    Decay ε linearly
end for

Output: $\pi^*(s) = \arg\max_a Q(s,a)$

Note (SARSA): replace $\max_{a'} Q(s_{t+1}, a')$ with $Q(s_{t+1}, a_{t+1})$ for on-policy updates.

5.7 Reference heuristics

The six reference rules are defined in Table 6 as implemented. None uses the WAIT action. They fall into three groups.

Rules accounting for queues and travel. The Minimum Truck Waiting Time rule (MTWT) sends the truck to the shovel where it will wait the least on arrival, combining the mean travel time and the predicted release of each shovel, trucks already en route included; it chooses the dump in the same way. It represents the rules of commercial fleet management systems [49]. The policy based on a linear program (LP) follows the DISPATCH principle [37]: a linear program sets, at the start of the shift, target flow rates on each shovel–dump route; at each decision, the truck is sent to the route that is furthest behind its plan. The program maximises production under capacity constraints on shovels and dumps, truck conservation and available truck-hours, limiting the planned utilisation of each resource to a value $u \leq 1$. With $u = 1$, the plan saturates the nearest shovel, which is optimal without randomness but creates an unbounded queue with random durations. The value $u = 0.7$ was chosen among seven candidates on the selection seeds, never on the evaluation seeds, and is used for all three regimes. These two rules use only mean quantities (travel times, service durations, breakdown probability), never the random events of the episode.

Rules accounting for queues only, or for no information. The FIFO rule first serves the shovel that has been waiting for a truck the longest (Section 2); if all shovels are busy, it chooses the one that will be free soonest, trucks en route included. It ignores travel: a shovel free in one minute but far away comes before a nearby shovel free in two minutes. Fixed Assignment ties each truck to one shovel and one dump for the whole shift, without reassignment.

Geometric rules. Nearest Shovel and Shortest Path ignore queues. As the decision is taken at the dump, they compare all shovels. But shovel 1 is the closest to the parking area and to both dumps: Nearest Shovel sends the whole fleet there for the whole shift, and its production is capped by the capacity of a single shovel (about 4,200 t/h). Shortest Path, whose cost also counts the shovel–dump trip, sends about 72% of its cycles to this shovel. These two rules are lower bounds, not representatives of industrial practice.

Table 6: Definition of the reference heuristics as implemented.

Policy	Rule applied when truck i becomes available
MTWT	For each shovel j: predicted arrival $a_j = t+$ mean empty travel time; predicted wait $\max(0,$ predicted release of $j - a_j)$. Choose the shovel with the smallest wait (ties: the earliest reached), then the dump in the same way, from the predicted end of loading.
LP (linear program)	At the start of the shift, target flow rates x_{jk} (cycles/h) maximising production, planned utilisation $\leq u = 0.7$. At each decision: the route whose number of dispatched cycles is furthest behind x_{jk}, at the predicted arrival time at the shovel; ties: the nearest shovel.
FIFO	Assign the truck to the shovel that has been waiting for a truck the longest (if all are busy, the one that will be free soonest), and likewise for the dump, regardless of distances.
Fixed Assignment	Assign truck i to shovel $i \bmod 3$ and dump $i \bmod 2$ for the whole shift, without reassignment.
Nearest Shovel	Assign the truck to the shovel closest to its position on the graph, then to the dump closest to that shovel, regardless of queues.
Shortest Path	Assign the truck to the shovel–dump pair that minimises the cost $C = 0.5\,T + 0.3\,D + 0.2\,E/6.49$ of the truck–shovel and shovel–dump trips, where T, D and E are the mean time, distance and fuel use computed with Dijkstra's algorithm; dividing by 6.49 cancels the effect of fuel calibration on decisions.

5.8 System architecture and computational cost

The architecture decouples the learning logic from the simulation, which makes it possible to test different agents interchangeably (Figs. 2 and 3). The agent selects an action a_t from the state s_t; the environment applies the assignment and returns the next state and the reward; the evaluation module computes the indicators for the ten policies (six heuristics and four agents) in the three regimes.

All measurements are made on the CPU of an 8-core laptop, without a graphics card.

- **Training:** a full training run under nominal conditions, with four training runs at the same time on the machine, took 223 min for PPO, 168 min for DQN, 253 min for Q-Learning and 241 min for SARSA. Each run uses a single simulated environment, as required by per-minute discounting (Section 5.4.3). Collecting with four environments in parallel would speed PPO up by a factor of only 1.47 and would change the resulting model (4,096 steps per update instead of 1,024): this option was not retained. Training is deterministic for a given seed. The full campaign comprises 90 training runs: 60 for the main results (4 algorithms, 3 regimes, 5 seeds) and 30 for the ablation study. It took 5 days and 9 hours, from 22 to 27 October 2026, four runs at a time, i.e. 515 cumulative training hours. During the campaign, as the machine was also used for other computations, the duration of a run varied from 142 to 1,322 min depending on the agent and the time; the durations above, measured during tuning, are more representative. The single evaluation (6,540 episodes) took 4 h 18 on 4 processes.
- **Evaluation:** each evaluation episode depends only on the policy and its seed, thanks to the separate random streams (Section 5.3). Episodes are therefore distributed over several processes, with exactly the same results as sequentially, seed by seed. On 150 episodes, 4 processes halve the duration; 8 processes do worse than 4, because each must first load PyTorch.
- **Inference:** the latency of a decision, state construction included, is below 1 millisecond for all policies: from 0.48 to 0.71 ms for PPO and from 0.36 to 0.49 ms for DQN, from 12 to 24 trucks. Since the state has a fixed size, it depends little on fleet size. It is negligible compared with the duration of a haulage cycle, but its suitability for the constraints of a commercial fleet management system has not been assessed.

- **Memory:** a PPO model takes 501 KB (plus 2 KB for the reward-normalisation statistics), a DQN model 344 KB, and a Q-Learning or SARSA table 0.6 to 1.3 MB depending on the number of states visited during training.

6 Experimental design

This section details the operating regimes, the training and evaluation protocol, the indicators and the statistical framework.

6.1 Operating regimes

Three regimes are defined (Table 7 and Fig. 4). Each episode corresponds to an 8-hour shift (480 minutes).

To situate each regime with respect to loading capacity, we use the match factor [8]:

$$\mathrm{MF} = \frac{N \times t_{\mathrm{load}}}{S \times t_{\mathrm{cycle}}}, \tag{10}$$

where N is the number of trucks, S the number of shovels, $t_{\mathrm{load}} = 2$ min the mean loading time and $t_{\mathrm{cycle}} \approx 22.7$ min the mean cycle duration excluding waiting and breakdowns, measured with Fixed Assignment. A factor below 1 indicates that shovels wait for trucks. It is about 0.35 under nominal conditions and 0.52 under high load: even with 18 trucks, the fleet remains below the capacity of the three shovels, which together can load about 12,600 t/h. The high-load regime therefore increases pressure on queues, but does not oversaturate the fleet; only a policy that concentrates trucks on a single shovel can saturate it.

Table 7: Parameters of the three operating regimes.

Regime	Trucks	Shovels	Dumps	Breakdowns per cycle	MF	Characteristic
Nominal	12	3	2	2%	0.35	Reference conditions
High load	18	3	2	2%	0.52	Fleet enlarged by 50%
Frequent breakdowns	12	3	2	10%	0.35	Frequent truck breakdowns

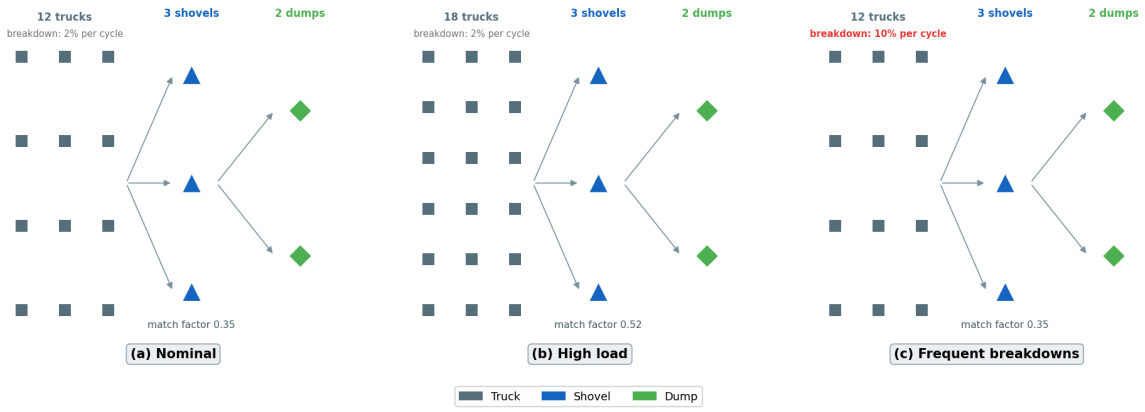


Figure 4: Configuration of the three operating regimes.

6.2 Training and evaluation protocol

Three disjoint sets of seeds. The evaluation seeds (42 to 71) provide the 30 replications of each result. The selection seeds (20,000 to 20,009) were used for every choice made before the campaign: hyperparameter tuning, choice of u for the LP policy and agent tuning (Section 6.3).

The training seeds (1 to 5) initialise the agents; the agent with seed s plays its training episodes on seeds $s \times 10^6, s \times 10^6 + 1, \ldots$, which never intersect the other two sets. A test checks, for the four algorithms, that all seeds used during training are training seeds. No choice was made on the evaluation seeds, and no agent was evaluated on them before the final campaign; nothing was changed after seeing its results.

Specialised agents. Each learning agent is trained in each of the three regimes with 5 training seeds: 2 million steps for PPO and DQN, 30,000 episodes for Q-Learning and SARSA, i.e. 60 training runs. The main results compare, in each regime, the agents trained in that regime with the six heuristics.

Evaluation without retraining. To measure the ability of a learned policy to withstand a change of regime, the agents trained under nominal conditions are also evaluated, without any retraining, under high load and frequent breakdowns, on the same seeds as the specialised agents. This is possible for the four agents, since the state has the same size whatever the number of trucks (Section 5.4). The nominal agents are also evaluated in two unseen intermediate regimes: 15 trucks with 2% breakdowns, and 12 trucks with 5% breakdowns.

Ablation study. Sensitivity to the weight of the waiting penalty is measured under nominal conditions for $w_4 \in \{0; 0.1; 0.5\}$, the retained value $w_4 = 0.3$ being that of the nominal agents. It covers PPO and DQN, with the same 5 training seeds, i.e. 30 additional training runs. PPO is the main agent of the study, and DQN the one whose behaviour depends most on the WAIT action (Section 6.3). Q-Learning and SARSA, whose convergence is insufficient, are excluded. This choice was fixed before the campaign.

Evaluation. All policies are evaluated on the same 30 replications. The agents choose the highest-valued action, without exploration: deterministic mode for PPO, $\varepsilon = 0$ for DQN and greedy choice for the tabular agents.

Two sources of variability. The 30 replications measure the variability due to the environment; the 5 training seeds measure that due to learning. Both are reported separately (Section 6.5).

6.3 Agent tuning

With a first tuning procedure (a grid of four configurations at 200,000 steps), the agents performed worse than a random policy. Several trials were then run to understand and improve learning. Multiplying trials until reinforcement learning outperforms the heuristics would be a bias: each trial is a chance to stumble on a favourable version by luck. This section therefore describes the rule followed and reports *all* trials, including those that failed.

Rule. It was written and dated before the trials it governs, and extended before each new step:

- all trials are run under nominal conditions, with training seed 1, and are judged on the mean productivity of the policy without exploration on the selection seeds; the evaluation seeds are never used;
- each step is fixed in advance, with its decision criterion, and a variant is retained only if it does better than the version it would replace;
- everything retained for PPO applies to DQN where it makes sense, and the tabular agents receive a tuning of the same size;
- the list of steps is closed: after the last one, the agents go into the campaign whatever their result, and this result is reported as is, even if the heuristics remain ahead.

Each trial is recorded with its configuration and the commit of the code used, and can be rerun with a single command.

Trials. Table 8 reports them in order. The reference points on the same seeds are: MTWT 4,443 t/h, LP 4,412, FIFO 4,246, Fixed Assignment 4,226, random policy 4,167, Nearest Shovel 3,915.

Table 8: Agent-tuning trials (productivity in t/h on selection seeds 20,000 to 20,009, nominal conditions, training seed 1). In brackets: the share of WAIT decisions or the share of cycles sent to a single shovel, when notable.

No.	Trial	Result	Decision
Per-decision discounting			
0	Tuning of PPO and DQN, 200,000 steps	PPO 3,953 (99% shovel 1); DQN 3,992	starting point
1, 2, 4	Reward estimated and paid at the decision, with or without a cost of queue minutes	PPO 3,915 to 3,981; DQN 4,106 to 4,191 (WAIT 52 to 65%)	rejected
3	PPO, full budget, reward paid at the end of the cycle	PPO 4,260	kept
5	Predicted waiting per route in the state (23 components)	PPO 4,345	retained
6	Normalised reward and $\gamma = 0.995$	PPO 4,405 (WAIT 36%)	retained
7	DQN check, full budget, $\gamma = 0.995$	DQN 3,593 (99% shovel 3)	kept, no variant
8, 9	Q-Learning and SARSA, initial configuration	Q-Learning 3,955 (62% shovel 3); SARSA 3,535 (98% shovel 3)	starting point
Opportunity cost extended to all cycles (abandoned)			
10–13	Same configuration, modified reward	PPO 210 (WAIT 99.8%); DQN 3,532 (WAIT 94.8%); Q-Learning 4,099; SARSA 4,134	abandoned
Per-minute discounting			
14–17	Same configuration as trials 6 to 9	PPO 4,414 (WAIT 0%); DQN 121 (WAIT 99.8%); Q-Learning 3,974; SARSA 3,602 (100% shovel 3)	PPO and DQN frozen
16–23	Tabular tuning: $\alpha \in \{0.1; 0.2\} \times \{6; 8\}$ intervals	Q-Learning 3,369 to **4,181** ($\alpha = 0.2$, 6 int.); SARSA 3,481 to **3,626** ($\alpha = 0.1$, 6 int.), 98 to 100% shovel 3	best configurations
24, 25	Sixth state variable: route of smallest predicted waiting	Q-Learning 4,062; SARSA 3,505 (95% shovel 3)	rejected

What was retained, and why. Each retained change is justified independently of its score. The block of predicted waiting per route gives the agent the same information as MTWT. Reward normalisation and a longer horizon are common practice for PPO. Per-minute discounting corrects the formulation of the problem (Section 5.4.3). Over the same period, the reference heuristics were strengthened: prediction of trucks en route for MTWT and FIFO, and addition of the LP policy. The comparison has therefore become harder for reinforcement learning.

The flaw found, and the abandoned variant. With per-decision discounting, the three value-based agents (DQN, Q-Learning, SARSA) drifted towards the farthest shovel, and their productivity decreased as exploration diminished. A measurement without retraining revealed the cause. Always going to shovel 3 rather than shovel 1 loses 9% of productivity, but hardly changes the reward per decision (0.813 versus 0.820), because a cycle yields 1 whatever its duration. A first correction, decided with the research supervisor, extended the opportunity cost to all cycles (cycle duration divided by 23.2 min). The reward of a shift changed only by a constant, and the drift disappeared (trials 10 to 13). But a cycle then yielded a negative value on average. With per-decision discounting, postponing a negative reward makes it less costly, and inserting waits of 0.2 min became the way to postpone it. PPO thus learned to wait in 99.8% of its decisions. Both flaws therefore have the same cause: per-decision discounting while decisions do not last the same time. This variant was abandoned, the reward returned to its previous definition, and per-minute discounting was introduced for the four algorithms, with no other change. These two decisions were taken after seeing results on the selection seeds: they are declared as such.

The collapse of DQN. With per-minute discounting, DQN chooses WAIT in 99.8% of its decisions (trial 15). Replayed greedily, the final model produces 840 t per shift, whereas the last training shifts, played with updates and exploration, produced 25,620 to 30,660 t. Our interpretation, consistent with this measurement but not demonstrated, is the following. A wait of 0.2 min delays the next production by only 0.2 min: its value is therefore almost equal to that of the best action, which is correct. The gap then becomes smaller than the approximation error of the network. As soon as this error puts waiting first, the model waits, and one wait leads to others. PPO, which

learns a policy rather than values to compare, is not affected. The equivalence test of Section 5.4.3 rules out an implementation error. In accordance with the rule, DQN goes into the campaign in this version.

The tabular agents. Tuning retained $\alpha = 0.2$ for Q-Learning and $\alpha = 0.1$ for SARSA, with 6 intervals in both cases. Giving the discrete state the route MTWT would choose, instead of the nearest shovel, cost both algorithms about 120 t/h (trials 24 and 25), and did not correct the concentration of SARSA on shovel 3. In the ten tabular trials with per-minute discounting (16 to 25), the productivity measured during training starts at about 4,160 t/h, when exploration is almost random, then decreases as exploration diminishes. The best tabular agent (4,181 t/h) barely matches the random policy (4,167 t/h). We present this as a convergence failure of the tabular approach on this problem, with the reduced state and budget retained, not as a limitation of these algorithms in general.

6.4 Indicators

In this study, operational resilience refers to the ability of a policy to maintain its throughput and contain queueing when the system moves from nominal to disrupted conditions. It is measured by the following indicators, computed for each replication.

1. **Productivity:** tonnage delivered per hour (t/h), and productivity per truck (t/h/truck), which makes it possible to compare regimes whose fleets differ in size.
2. **Queueing time:** total time spent by a truck queueing at shovels and dumps during the shift, averaged over trucks (min per truck per shift). It is not a waiting time per cycle: with about 20 cycles per truck under nominal conditions, 11 min of queueing per shift correspond to about 0.6 min per cycle.
3. **Total idle time:** sum, per truck and per shift, of queueing, breakdown immobilisation and time spent waiting voluntarily through the WAIT action.
4. **Truck utilisation:** share of the shift during which trucks travel, load or dump (%). The active time of the last cycle that runs past the end of the shift is excluded.
5. **Shovel utilisation:** share of the shift during which shovels load (%).
6. **Specific consumption:** fuel consumed per tonne delivered (L/t).
7. **Number of completed cycles** and **cumulative reward** with the reference weights.
8. **Robustness ratio:** ratio between the performance in a disrupted regime and the nominal performance:

$$\mathrm{RR}(p, s) = \frac{\mathrm{Performance}(p, s)}{\mathrm{Performance}(p, \mathrm{nominal})}. \tag{11}$$

   Under high load, this ratio is computed on productivity per truck; otherwise it would mainly reflect the added trucks.

6.5 Statistical protocol

Statistical unit. For paired comparisons, the unit of analysis is the replication: each (policy, regime) pair provides exactly 30 values, one per evaluation seed. For an agent, the value of a replication is the mean of its 5 models (one per training seed) on that seed.

Pairing. The same seed is used by all policies in a given replication, and the simulator uses a separate random stream per process and one breakdown stream per truck (Section 5.3). Two policies played with the same seed therefore face the same random events: in particular, the k-th cycle of a given truck has the same breakdown, of the same duration, whatever the policy. Observations are paired by seed, and replications are treated as blocks. Travel and service times are still drawn in the order of events, which depends on the decisions: the pairing is strong, but not perfect.

Tests. In each regime, the Friedman test [15] checks for an effect of the policy. Pairwise comparisons use the two-sided Wilcoxon signed-rank test [48], with a Holm correction [19] over the family of comparisons reported in each regime [12]. These comparisons were fixed before the campaign: each of the four agents is compared with MTWT, the best heuristic on the selection seeds, and with Fixed Assignment, a widespread industrial reference (Section 2), i.e. eight comparisons per regime and per indicator. Regime dependence is tested on the three main regimes only. The dependence of the ranking on the regime (H2) is tested with a linear model of productivity per truck including the policy, the blocks (regime, seed) and the interaction between policy and regime.

Effect sizes and intervals. Differences are reported in physical units, with a 95% paired bootstrap confidence interval (10,000 resamples of the per-seed differences) [13]. Cohen's d is reported for descriptive purposes; in a simulator, variability between replications is low, so a large d does not mean a large practical effect. p-values below 0.001 are reported as $p < 0.001$.

Training variability. For each agent and each regime, the score of a model is its mean over the 30 evaluation seeds. The 5 models are summarised by their interquartile mean (IQM), with a 95% confidence interval from a bootstrap stratified by regime, and by the worst and best model [3]. The probability of improvement $P(X > Y)$ between two agents is also reported. For the aggregate over the three regimes, scores are divided by the mean of Fixed Assignment in the same regime. Learning curves are the mean of the 5 evaluated models, with a pointwise bootstrap interval; they are read from the training logs of these models, not from separate training runs.

6.6 Reproducibility parameters

The simulation is implemented in Python 3.11.11 with Gymnasium 1.2.3 [44], Stable-Baselines3 2.8.0 [31] and PyTorch 2.5.1, on CPU. The reward weights ($w_1 = 1$, $w_2 = 0.1$, $w_3 = 0.05$, $w_4 = 0.3$) are constant in all experiments, except in the ablation study. The per-minute discount factor is derived from γ and is never tuned (Section 5.4.3). The per-replication results, one row per policy, regime and seed, are provided with the code.

All agents of this version are trained during the same campaign, with the same simulator and the same reward. Each model comes with a metadata file (code version, hyperparameters, training time), the configuration used is archived with the results, and each tuning trial can be rerun with a single command.

7 Results

This section presents productivity, queueing, policy behaviour, trade-offs between criteria, robustness and transfer without retraining, then statistical significance, variability, convergence and the ablation study. All values come from the single evaluation of the final campaign, on seeds 42 to 71. For an agent, each replication is first averaged over its 5 models (training seeds): the reported mean and standard deviation then refer to the 30 replications, as for a heuristic. Variability between models is reported separately (Table 17 and Section 7.7).

7.1 Productivity

Table 9 reports the mean productivity of the ten policies in the three regimes, as well as productivity per truck.

Table 9: Productivity (mean ± standard deviation over 30 replications; for an agent, mean of its 5 models) and productivity per truck. N: nominal; HL: high load; FB: frequent breakdowns. In bold: the highest in each regime.

	Productivity (t/h)			Per truck (t/h/truck)		
Policy	**N**	**HL**	**FB**	**N**	**HL**	**FB**
FIFO	4,244.3 ± 51	6,224.2 ± 68	4,000.5 ± 79	353.7	345.8	333.4
Fixed Assignment	4,246.1 ± 45	6,204.9 ± 49	3,999.3 ± 69	353.8	344.7	333.3
Nearest Shovel	3,893.2 ± 50	4,023.2 ± 39	3,754.3 ± 69	324.4	223.5	312.9
Shortest Path	4,026.2 ± 47	4,657.9 ± 177	3,857.6 ± 57	335.5	258.8	321.5
MTWT	**4,437.4 ± 55**	**6,365.3 ± 64**	**4,183.7 ± 79**	**369.8**	**353.6**	**348.6**
LP	4,404.2 ± 70	6,351.3 ± 66	4,149.8 ± 88	367.0	352.9	345.8
Q-Learning	4,088.2 ± 47	5,787.9 ± 55	3,839.8 ± 73	340.7	321.6	320.0
SARSA	3,573.3 ± 55	5,409.8 ± 47	3,353.1 ± 65	297.8	300.5	279.4
DQN	1,287.0 ± 79	984.5 ± 10	1,578.5 ± 64	107.2	54.7	131.5
PPO	4,414.7 ± 51	6,310.0 ± 62	4,163.9 ± 77	367.9	350.6	347.0

The same three policies lead in all three regimes. MTWT is the most productive policy in every regime: 4,437 t/h under nominal conditions, 6,365 t/h under high load and 4,184 t/h under frequent breakdowns. PPO and LP follow closely, within 1%: PPO is 23 t/h (−0.5%), 55 t/h (−0.9%) and 20 t/h (−0.5%) below MTWT. Next come FIFO and Fixed Assignment, almost tied, 2 to 4% below MTWT depending on the regime. No learning agent outperforms the best heuristic, in any regime.

The other agents are below Fixed Assignment. Q-Learning loses 158 to 417 t/h against Fixed Assignment depending on the regime, and SARSA 646 to 795 t/h. DQN produces 985 to 1,579 t/h on average: four of its five nominal models collapse onto the WAIT action (Sections 7.3 and 7.7).

The distance rules hit a ceiling. Nearest Shovel and Shortest Path send most of the fleet to the nearest shovel (Section 7.3). Under high load, their productivity per truck drops by 31% and 23%, against 2 to 6% for the policies that spread the fleet.

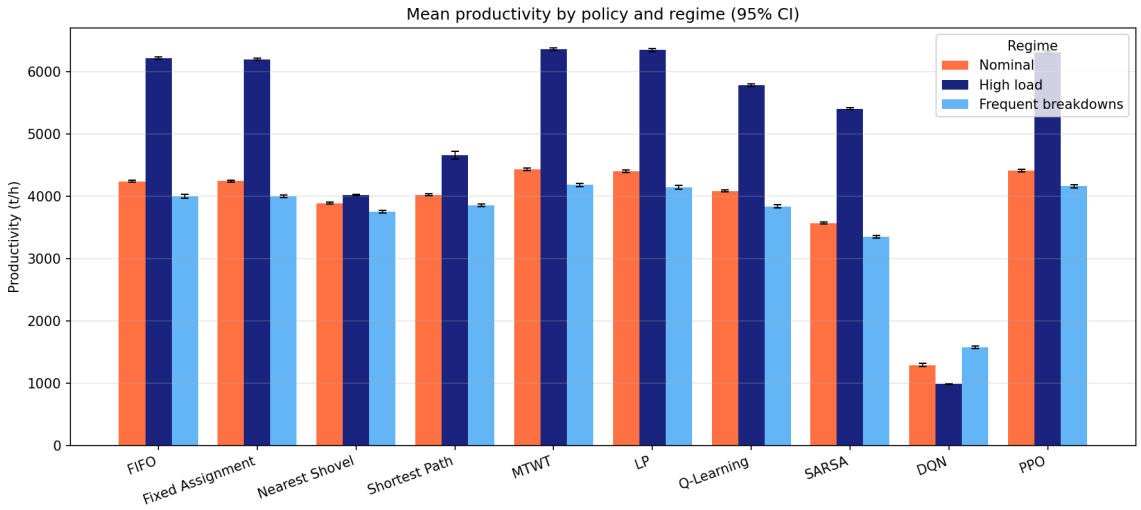


Figure 5: Mean productivity by policy and regime. Error bars show 95% bootstrap confidence intervals over the 30 replications.

Table 10 reports the number of completed cycles and the cumulative reward. The ranking by reward follows the productivity ranking, except for one swap between two very close policies under nominal conditions (FIFO and Fixed Assignment) and under high load (PPO and FIFO). The reward of DQN is negative: its repeated waits accumulate the WAIT penalty and the opportunity cost.

Table 10: Mean number of completed cycles per shift and cumulative reward, over 30 replications (N: nominal, HL: high load, FB: frequent breakdowns).

Policy	Completed cycles			Cumulative reward		
	N	**HL**	**FB**	**N**	**HL**	**FB**
FIFO	242.5	355.7	228.6	220.6	322.6	208.0
Fixed Assignment	242.6	354.6	228.5	219.8	320.6	207.0
Nearest Shovel	222.5	229.9	214.5	192.5	152.0	186.9
Shortest Path	230.1	266.2	220.4	199.4	196.1	192.2
MTWT	**253.6**	**363.7**	**239.1**	**230.9**	**330.3**	**217.7**
LP	251.7	362.9	237.1	227.0	329.7	213.9
Q-Learning	233.6	330.7	219.4	200.5	281.0	190.1
SARSA	204.2	309.1	191.6	175.9	265.1	158.7
DQN	73.5	56.3	90.2	−148.6	−417.4	−105.5
PPO	252.3	360.6	237.9	228.3	320.9	215.5

7.2 Queueing and idle time

Table 11 distinguishes queueing time, which measures congestion, from voluntary waiting (WAIT action) and total idle time, which adds repairs. All are expressed in minutes per truck per shift.

Table 11: Queueing time (mean ± standard deviation over 30 replications), voluntary waiting (WAIT) and total idle time (queue + repair + WAIT), in minutes per truck per shift.

	Nominal			High load			Frequent breakdowns		
Policy	**Queue**	**WAIT**	**Total**	**Queue**	**WAIT**	**Total**	**Queue**	**WAIT**	**Total**
FIFO	12.9 ± 1.7	0.0	20.6	21.5 ± 2.6	0.0	29.2	11.1 ± 1.9	0.0	46.5
Fixed Assignment	11.3 ± 1.4	0.0	18.9	19.0 ± 2.1	0.0	26.7	10.6 ± 1.3	0.0	45.9
Nearest Shovel	62.1 ± 5.5	0.0	69.3	188.3 ± 8.0	0.0	193.9	52.8 ± 5.2	0.0	86.3
Shortest Path	61.4 ± 5.4	0.0	68.7	152.9 ± 14.8	0.0	159.0	52.1 ± 5.1	0.0	86.4
MTWT	12.2 ± 1.3	0.0	20.2	21.4 ± 2.4	0.0	29.2	10.8 ± 1.3	0.0	47.5
LP	18.9 ± 1.9	0.0	27.0	21.1 ± 1.4	0.0	28.9	18.3 ± 1.6	0.0	54.9
Q-Learning	18.8 ± 1.1	9.6	35.8	37.6 ± 2.0	8.0	52.8	20.4 ± 1.5	7.1	61.6
SARSA	36.8 ± 1.9	0.2	43.8	45.6 ± 2.1	0.1	52.5	36.6 ± 2.2	5.6	72.1
DQN	19.5 ± 1.3	319.5	341.4	15.2 ± 0.3	395.0	411.5	16.9 ± 1.4	273.0	304.6
PPO	16.7 ± 1.1	0.0	24.7	22.6 ± 1.2	5.5	35.9	14.2 ± 1.1	0.0	50.9

Congestion. Fixed Assignment has the lowest queueing time among the policies that produce normally, in all three regimes (11.3, 19.0 and 10.6 min), closely followed by MTWT and FIFO. PPO queues slightly longer than MTWT (16.7 versus 12.2 min under nominal conditions) for almost the same productivity. The two distance rules accumulate 52 to 188 min of queueing, because they concentrate the fleet in front of one shovel.

Voluntary waiting. The six heuristics never use WAIT. PPO hardly uses it, except under high load, where it accounts for 26% of its decisions but only 5.5 min per truck, as these waits are short. Q-Learning waits in 33 to 46% of its decisions (7 to 10 min per truck). The trucks of DQN spend 273 to 395 min per shift in voluntary waiting, i.e. 57 to 82% of the shift: this is the collapse described in Section 6.3. Their low queueing time is therefore not a sign of efficiency: these trucks hardly move.

Frequent breakdowns. Queueing is slightly lower than under nominal conditions for most policies, since fewer trucks are available at the same time. The increase in total idle time (about 26 to 28 min per truck for the heuristics that spread the fleet) comes from repairs.

7.3 Allocation behaviour of the policies

Table 12 describes the allocation of cycles among the three shovels and the share of decisions devoted to the WAIT action. The allocation among shovels is not recorded by the evaluation: it is obtained

by replaying the same episodes with the cycle log. Each replayed episode was checked to be identical to the evaluated one (productivity, queueing time and fuel); nothing was chosen on these episodes.

Table 12: Allocation of cycles among shovels 1, 2 and 3 (%) and share of WAIT decisions (%), means over 30 replications (and over the 5 models of an agent).

	Nominal		High load		Frequent breakdowns	
Policy	**S1 / S2 / S3**	**WAIT**	**S1 / S2 / S3**	**WAIT**	**S1 / S2 / S3**	**WAIT**
FIFO	38 / 33 / 29	0	38 / 33 / 29	0	38 / 33 / 29	0
Fixed Assignment	36 / 33 / 31	0	36 / 33 / 31	0	36 / 34 / 31	0
Nearest Shovel	100 / 0 / 0	0	100 / 0 / 0	0	100 / 0 / 0	0
Shortest Path	74 / 26 / 0	0	71 / 29 / 0	0	72 / 28 / 0	0
MTWT	50 / 35 / 15	0	42 / 36 / 22	0	49 / 37 / 14	0
LP	60 / 40 / 0	0	42 / 42 / 17	0	64 / 36 / 0	0
Q-Learning	36 / 34 / 31	42	39 / 28 / 34	46	36 / 34 / 30	33
SARSA	0 / 0 / 99	2	1 / 30 / 69	1	0 / 1 / 99	15
DQN	46 / 27 / 27	79	44 / 15 / 42	99	50 / 43 / 7	96
PPO	59 / 36 / 4	0	46 / 36 / 18	26	56 / 37 / 7	0

Three profiles emerge among the heuristics. The distance rules concentrate the fleet: Nearest Shovel sends all its cycles to shovel 1, and Shortest Path 71 to 74%. FIFO and Fixed Assignment split the cycles almost equally, and therefore send nearly a third of them to shovel 3, the farthest. MTWT and LP favour shovels 1 and 2 and use shovel 3 only when the other two are busy, mostly under high load (22 and 17%). PPO follows the same profile as MTWT, but uses shovel 3 even less: 4% against 15% under nominal conditions, 7% against 14% under frequent breakdowns. Its cycles are therefore shorter, which explains its lower consumption per tonne (Section 8.4).

The value-based agents show extreme behaviours. SARSA sends 99% of its cycles to shovel 3 under nominal conditions and frequent breakdowns, as during tuning. Q-Learning spreads its cycles over the three shovels, but waits in 33 to 46% of its decisions. DQN waits in 79 to 99% of its decisions. Its only working nominal model (seed 2, 3,932 t/h) sends 99% of its cycles to shovel 1: it reproduces Nearest Shovel (3,893 t/h), not a new policy.

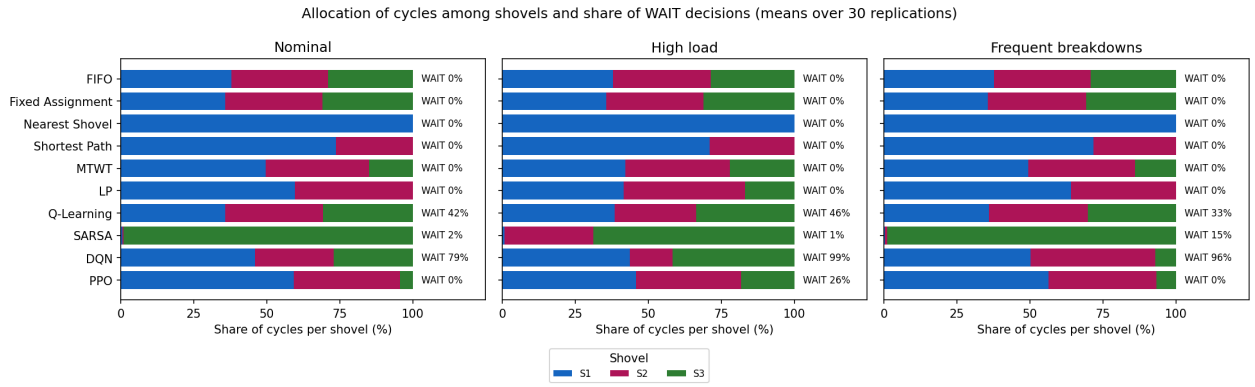


Figure 6: Allocation of cycles among shovels (stacked bars) and share of WAIT decisions, by policy and regime, means over 30 replications (and over the 5 models of a specialised agent).

7.4 Trade-offs between productivity, queueing and fuel

Table 13 gives, for each regime, the Pareto non-dominated policies for two pairs of criteria: productivity and queueing time, then productivity and fuel per tonne.

Table 13: Pareto non-dominated policies in each regime. DQN, non-dominated under high load on the first pair because its trucks hardly move, is shown in brackets.

Regime	Productivity and queueing time	Productivity and fuel per tonne
Nominal	Fixed Assignment (4,246 t/h; 11.3 min), MTWT (4,437; 12.2)	Nearest Shovel (3,893 t/h; 0.235 L/t), LP (4,404; 0.235), PPO (4,415; 0.237), MTWT (4,437; 0.244)
High load	Fixed Assignment (6,205; 19.0), LP (6,351; 21.1), MTWT (6,365; 21.4), (DQN: 985; 15.2)	PPO (6,310; 0.248), MTWT (6,365; 0.253)
Frequent breakdowns	Fixed Assignment (3,999; 10.6), MTWT (4,184; 10.8)	Nearest Shovel (3,754; 0.235), LP (4,150; 0.236), PPO (4,164; 0.242), MTWT (4,184; 0.245)

Between productivity and queueing time, the trade-off is minimal: MTWT produces 4.5% more than Fixed Assignment for less than one extra minute of queueing per truck; together with Fixed Assignment, and LP under high load, it dominates all other policies. Between productivity and fuel, by contrast, several policies form the front: PPO and LP use less fuel per tonne than MTWT, for slightly lower productivity. Under high load, PPO is the most fuel-efficient policy per tonne. These fronts are defined within each regime.

7.5 Robustness and transfer without retraining

The robustness ratios of Table 14 compare, for the agents, models trained in each regime: they measure performance after retraining. The ability of the same model to withstand a change of regime is measured by the evaluation without retraining (Table 15).

The queueing increase index in regime s is defined as:

$$\Delta^{s}_{\text{queue}} = \frac{W_s - W_{\text{nominal}}}{W_{\text{nominal}}} \times 100, \tag{12}$$

where W is the mean queueing time per truck per shift.

Table 14: Robustness with respect to nominal conditions. Under high load (HL), the ratio is computed on productivity per truck; under frequent breakdowns (FB), on productivity. Increases are computed on queueing time and, for comparison, on total idle time.

	Robustness ratio		Δ_{queue} (%)		Total idle
Policy	**HL (per truck)**	**FB**	**HL**	**FB**	**HL (%)**
FIFO	0.978	0.943	+67.5	−13.8	+41.7
Fixed Assignment	0.974	0.942	+68.8	−6.2	+40.8
Nearest Shovel	0.689	0.964	+203.4	−14.9	+179.9
Shortest Path	0.771	0.958	+148.8	−15.1	+131.5
MTWT	0.956	0.943	+75.5	−11.4	+44.0
LP	0.961	0.942	+12.0	−2.9	+7.0
Q-Learning	0.944	0.939	+100.3	+8.8	+47.3
SARSA	1.009	0.938	+23.8	−0.5	+19.9
DQN	0.510	1.227	−21.9	−13.3	+20.5
PPO	0.953	0.943	+35.6	−15.1	+45.1

Under high load, the policies that spread the fleet keep 94 to 98% of their productivity per truck; the two distance rules keep only 69 and 77%. Under frequent breakdowns, all policies that work lose 4 to 6% of productivity, and their ratios are almost identical (0.938 to 0.964): truck breakdowns

affect all policies in the same way. The ratios of DQN (0.510 and 1.227) and of SARSA under high load (1.009) mainly reflect very low or highly model-dependent nominal levels, not robustness.

Table 15: Evaluation without retraining: agents trained under nominal conditions (transferred) and agents trained in the evaluated regime (specialised), on the same seeds. Productivity in t/h, mean of the 5 models; Δ = transferred − specialised, with a 95% paired bootstrap confidence interval over the 30 replications.

Regime	Agent	Specialised	Transferred	Δ (t/h) [95% CI]	Δ (%)
Frequent breakdowns	PPO	4,163.9	4,163.5	−0.5 [−6.2; 5.0]	0.0
	DQN	1,578.5	1,226.2	−352.3 [−368.0; −329.2]	−22.3
	Q-Learning	3,839.8	3,871.6	+31.7 [22.9; 40.4]	+0.8
	SARSA	3,353.1	3,419.2	+66.0 [51.7; 79.5]	+2.0
High load	PPO	6,310.0	6,257.9	−52.1 [−67.1; −37.6]	−0.8
	DQN	984.5	1,802.3	+817.7 [724.7; 908.4]	+83.1
	Q-Learning	5,787.9	5,757.0	−30.9 [−49.6; −10.8]	−0.5
	SARSA	5,409.8	4,035.7	−1,374.1 [−1,393.0; −1,355.1]	−25.4

PPO transfers almost without loss. Under frequent breakdowns, the nominal PPO performs exactly like the specialised PPO (−0.5 t/h, interval containing 0). Under high load, it loses 0.8%; it remains ahead of Fixed Assignment (+53 t/h [28; 78]) and behind MTWT (−108 t/h [−130; −85]). It never uses WAIT, whereas the specialised PPO under high load uses it in 26% of its decisions. In the two unseen intermediate regimes, the nominal PPO likewise ranks just behind MTWT: −61 t/h with 15 trucks, −19 t/h with 5% breakdowns, and ahead of Fixed Assignment by 118 and 165 t/h.

The other agents. Q-Learning transfers with a gap below 1%. SARSA transfers well under frequent breakdowns but loses 25% under high load. The transferred nominal DQN does better than the specialised DQN under high load (+83%), because the specialised DQN collapses even more: this gap does not measure a transfer capability.

Except for SARSA under high load, retraining an agent in the disrupted regime therefore brings almost nothing in this study: truck breakdowns and a larger fleet do not change the dispatching logic a nominal agent has learned.

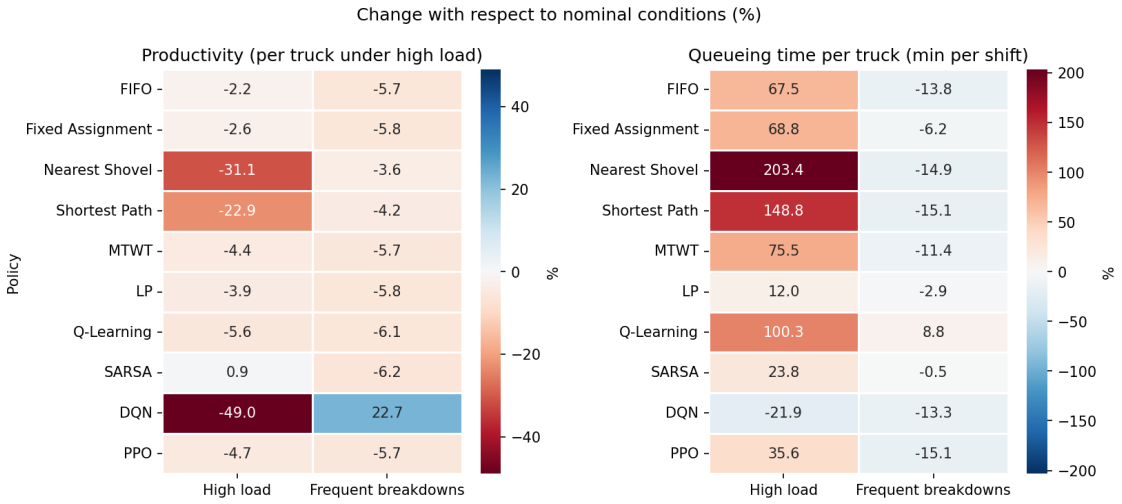


Figure 7: Change with respect to nominal conditions, in %. Left: productivity (per truck under high load). Right: queueing time. Blue indicates an improvement and red a degradation.

7.6 Statistical significance

The Friedman test shows an effect of the policy on productivity in each of the three regimes ($\chi^2(9) = 263.4$ under nominal conditions, 263.0 under high load and 263.2 under frequent breakdowns;

$p < 0.001$). The linear model of productivity per truck shows a significant interaction between policy and regime ($F(18, 783) = 1{,}211.1$, $p < 0.001$, $\eta_p^2 = 0.97$).

Table 16 reports the eight comparisons fixed before the campaign (Section 6.5). All are significant after Holm correction ($p < 0.001$). PPO is significantly below MTWT in all three regimes, by 20 to 55 t/h, and significantly above Fixed Assignment, by 105 to 169 t/h. The three other agents are below both references in all regimes. Cohen's d is very large for DQN and the tabular agents ($|d| > 2$): it reflects differences of several hundred t/h against low variability between replications. For PPO against MTWT, d ranges from -0.25 to -0.88: the gap is real, but small in practice (less than 1% of productivity).

Table 16: Pairwise comparisons of productivity, fixed before the campaign: mean paired difference (t/h) with 95% bootstrap confidence interval, and Cohen's d. Two-sided paired Wilcoxon test with Holm correction over the eight comparisons of each regime: $p < 0.001$ for all.

Comparison	Nominal Difference [95% CI]	d	High load Difference [95% CI]	d	Frequent breakdowns Difference [95% CI]	d
PPO − MTWT	−22.8 [−31.2; −14.2]	−0.43	−55.3 [−75.4; −35.5]	−0.88	−19.7 [−28.0; −11.2]	−0.25
DQN − MTWT	−3,150.5 [−3,177.0; −3,113.3]	−46.0	−5,380.8 [−5,403.5; −5,357.9]	−117.4	−2,605.2 [−2,629.0; −2,582.0]	−36.1
Q-Learning − MTWT	−349.2 [−366.3; −332.0]	−6.79	−577.4 [−596.9; −557.0]	−9.70	−343.8 [−357.0; −331.1]	−4.51
SARSA − MTWT	−864.2 [−879.6; −848.4]	−15.7	−955.5 [−971.4; −938.0]	−17.0	−830.6 [−848.2; −811.4]	−11.4
PPO − Fixed	+168.6 [158.1; 179.0]	3.52	+105.1 [86.9; 122.9]	1.89	+164.6 [152.4; 177.2]	2.24
DQN − Fixed	−2,959.1 [−2,984.3; −2,922.2]	−45.9	−5,220.4 [−5,238.3; −5,201.6]	−147.5	−2,420.8 [−2,445.3; −2,398.7]	−36.3
Q-Learning − Fixed	−157.9 [−172.7; −144.0]	−3.43	−417.0 [−438.8; −394.0]	−8.03	−159.5 [−170.0; −149.3]	−2.24
SARSA − Fixed	−672.8 [−688.7; −657.1]	−13.4	−795.1 [−810.3; −780.4]	−16.5	−646.2 [−664.4; −628.4]	−9.61

These comparisons concern the expected performance of each algorithm: for each replication, an agent is represented by the mean of its 5 models. They do not say that a model drawn at random would do the same: for DQN, one model in five reproduces Nearest Shovel and the others collapse (Tables 12 and 17).

7.7 Variability between replications and between models

Between replications. For a given model, the coefficient of variation of productivity over the 30 replications is low for all policies that work: 0.8 to 2.5% depending on the policy and the regime. It exceeds 3% only for Shortest Path under high load (3.8%) and for DQN (3.2 to 12.5%).

Between models. Table 17 gives the productivity of each of the 5 models of each agent. PPO is highly reproducible: its 5 models are within 10 t/h of one another under nominal conditions, 21 t/h under frequent breakdowns and 211 t/h under high load. The tabular agents vary more (up to 436 t/h apart). DQN is bimodal: in each regime, one or two models produce 2,900 to 4,200 t/h and the others less than 1,200 t/h, most of them less than 400 t/h. The mean of DQN therefore describes a mixture of working and collapsed models, not a typical level.

Table 17: Productivity of each of the 5 models of each agent (training seeds 1 to 5, in order), in t/h, mean over the 30 replications.

Agent	Nominal	High load	Frequent breakdowns
Q-Learning	4,171 / 4,057 / 4,123 / 4,096 / 3,994	5,776 / 5,928 / 5,919 / 5,521 / 5,796	3,861 / 3,820 / 3,837 / 3,874 / 3,807
SARSA	3,601 / 3,493 / 3,577 / 3,606 / 3,589	5,322 / 5,495 / 5,394 / 5,201 / 5,637	3,418 / 3,369 / 3,372 / 3,456 / 3,150
DQN	194 / 3,932 / 934 / 210 / 1,164	52 / 315 / 75 / 4,166 / 315	210 / 210 / 1,049 / 3,484 / 2,940
PPO	4,414 / 4,418 / 4,416 / 4,409 / 4,416	6,375 / 6,322 / 6,365 / 6,324 / 6,164	4,162 / 4,176 / 4,157 / 4,155 / 4,170

Aggregated as in Agarwal et al. [3], normalised by Fixed Assignment and pooled over the three regimes, the IQMs of the agents are 1.037 for PPO [1.035; 1.037], 0.956 for Q-Learning [0.949; 0.961], 0.850 for SARSA [0.842; 0.857] and 0.187 for DQN [0.065; 0.438]. The probability that a PPO model does better than a DQN or Q-Learning model in the same regime is 1.

7.8 Convergence

Figure 8 shows the productivity of the training episodes, averaged over the 5 models with a 95% confidence interval. These episodes are played with exploration, so the curve of a value-based agent (DQN, Q-Learning, SARSA) does not coincide with the performance of its greedy policy evaluated above.

- **PPO** improves from the start and reaches a plateau at about 10% of the budget, then remains stable: under nominal conditions, from 4,243 t/h in the first interval to 4,409 t/h at 10% and 4,423 t/h at the end. The same holds in the two disrupted regimes.
- **DQN** starts at the level of the random policy, when exploration is total (about 4,170 t/h under nominal conditions), then declines as exploration decreases, down to about 3,460 t/h at the end. Played without exploration, the final model does much worse (1,287 t/h on average): during training, the rare random actions are enough to get trucks out of repeated waits.
- **Q-Learning and SARSA** also decline as exploration decreases. Q-Learning falls from 4,169 to about 4,060 t/h halfway, then recovers slightly (4,108 t/h at the end). SARSA declines until the end (3,559 t/h). Neither exceeds the level reached at the start with almost total exploration, i.e. that of the random policy. This convergence failure, already observed during tuning, is confirmed over the 5 models and the three regimes.

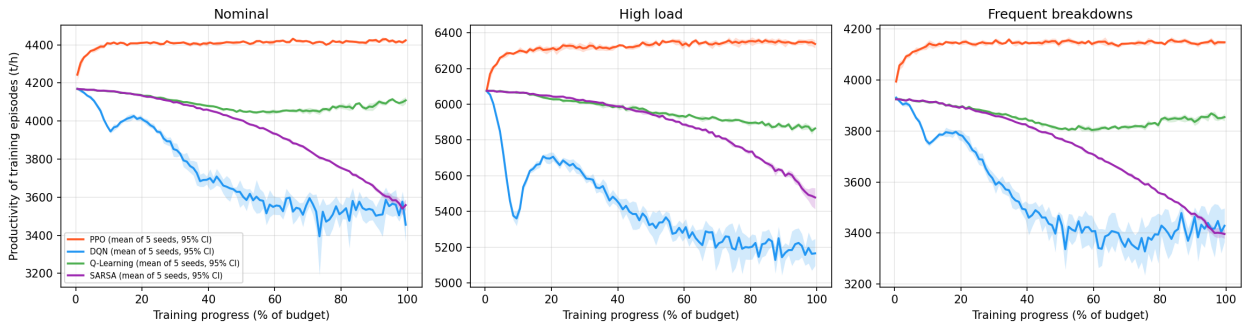


Figure 8: Productivity of the training episodes (exploration included) as a function of training progress, mean over the 5 models with a 95% confidence interval, by agent and regime.

7.9 Ablation study

The ablation study concerns the weight of the waiting penalty w_4, under nominal conditions, with 5 models per value for PPO and DQN (Table 18). The row $w_4 = 0.3$ uses the nominal models of the main results.

Table 18: Ablation study on the weight of the waiting penalty w_4, nominal regime, 5 models per value. Mean productivity (t/h), range of the 5 models, share of WAIT decisions (%), and for DQN the number of collapsed models (productivity below 1,500 t/h).

	PPO			DQN			
w_4	**Productivity**	**Range**	**WAIT (%)**	**Productivity**	**Range**	**WAIT (%)**	**Collapsed**
0	4,416.9	4,411 – 4,423	0.6	1,587.7	210 – 3,785	67.1	3 / 5
0.1	4,410.8	4,405 – 4,417	4.3	2,267.4	210 – 3,789	66.4	2 / 5
0.3 (ref.)	4,414.7	4,409 – 4,418	0.0	1,287.0	194 – 3,932	79.3	4 / 5
0.5	4,411.4	4,401 – 4,421	0.1	2,016.9	0 – 3,720	80.7	2 / 5

PPO is insensitive to w_4: its productivity only varies from 4,411 to 4,417 t/h, and it almost never uses WAIT. DQN collapses for every value of w_4, including without a waiting penalty, with 2 to 4 collapsed models out of 5. With $w_4 = 0.5$, two models wait for the whole shift and haul no

tonne at all; their fuel per tonne is undefined. The collapse of DQN therefore does not come from the weight of the waiting penalty, which is consistent with the explanation proposed in Section 6.3: it stems from the very small value gap between WAIT and the best assignment.

Sensitivity to hyperparameters. The hyperparameters of PPO and DQN were chosen on a grid of four configurations each, and those of the tabular agents on a grid of the same size, following the procedure of Section 6.3. The corresponding trials are reported in Table 8. No other hyperparameter sensitivity was measured.

8 Discussion

This section interprets the results in light of the research questions, discusses the implications for operations and sustainability, and then the limitations of the study.

8.1 Answers to the research questions

RQ1. MTWT achieves the highest productivity in all three regimes, and the top of the ranking is stable: MTWT, then PPO and LP within 1%, then FIFO and Fixed Assignment. What changes from one regime to another is the bottom of the ranking: the distance rules collapse under high load, and the poorly performing agents (DQN, SARSA) vary strongly. The significant interaction between policy and regime comes from these policies, not from a change at the top.

RQ2. Productivity and queueing go together. The most productive policies are also those that queue the least, and the Pareto front reduces to Fixed Assignment and MTWT (and LP under high load), with a gap of less than one minute of queueing per truck. What separates the policies is how they take queues into account: the policies that ignore them (distance rules) or learn poorly to use them (tabular agents, DQN) are dominated everywhere.

RQ3. Yes for PPO: the nominal model, deployed without retraining, does as well as the specialised model under frequent breakdowns and loses 0.8% under high load. It keeps its rank, just behind MTWT and ahead of Fixed Assignment, in the two disrupted regimes and in the two unseen intermediate regimes. Q-Learning also transfers, at its own low level. SARSA loses 25% under high load.

RQ4. No. The ranking by fuel per tonne does not follow the productivity ranking. Nearest Shovel, LP and PPO use the least fuel per tonne under nominal conditions and frequent breakdowns, and PPO the least under high load. FIFO and Fixed Assignment, although among the most productive, use 8 to 14% more fuel per tonne than PPO depending on the regime.

Table 19 summarises the status of the three hypotheses.

Table 19: Validation of the research hypotheses, formulated before the analysis of the results.

Hyp.	Statement	Key evidence	Verdict
H1	Under nominal conditions, the deep agents outperform the best heuristic in productivity.	PPO is −22.8 t/h (−0.5%) below MTWT, DQN −3,150 t/h; $p < 0.001$ after correction (Table 16). Mean over 5 models per agent.	**Refuted**
H2	No policy dominates in all three regimes.	MTWT is the most productive in all three regimes. The interaction between policy and regime is significant ($F(18, 783) = 1{,}211$, $p < 0.001$), but it comes from the bottom of the ranking.	**Refuted**
H3	High productivity does not necessarily come with low queueing time.	Front reduced to Fixed Assignment and MTWT (and LP under high load), with less than one minute of queueing difference. A clear trade-off does exist, however, between productivity and fuel per tonne (additional analysis, not covered by the hypothesis).	**Not supported**

8.2 Interpretation

- **Knowing the queues is what matters.** The three leading policies all use an estimate of future waiting: MTWT chooses the route with the smallest predicted waiting, LP follows a flow plan computed on capacities, and PPO observes the predicted waiting per route, added to the state during tuning (Section 6.3). It is therefore consistent that PPO almost reaches the level of MTWT: it has the same information, and does no better than the rule that exploits it directly. This reading is an interpretation; no experiment of the study removes this information from PPO once tuning was over.
- **PPO trades a little productivity for fuel.** PPO's reward penalises the distance travelled (w_3). PPO uses 1.4 to 2.8% less fuel per tonne than MTWT, for 0.5 to 0.9% less productivity, by sending fewer cycles to the farthest shovel (Table 12): a trade-off consistent with this penalty, which MTWT, which only looks at waiting, does not make.
- **The failure of DQN is reproducible and does not depend on w_4.** Four nominal models out of five collapse onto the WAIT action, and the collapse persists for every value of w_4, including without a waiting penalty. During training, exploration masks this collapse: training episodes produce about 3,460 t/h, the greedy policy 1,287 t/h on average. With a single model per agent, the result of DQN would have depended on the seed: 3,932 t/h with seed 2, whose model reproduces Nearest Shovel, 194 t/h with seed 1.
- **The tabular agents do not converge.** Their productivity decreases as exploration diminishes, in the three regimes and for the 5 models. A representation of six discretised variables does not seem sufficient to distinguish the situations in which a detour to another shovel pays off.
- **Evaluating across several regimes remains necessary,** even though the top of the ranking does not change here: it is under high load that the distance rules and SARSA lose the most, and an evaluation limited to nominal conditions would not have shown it.

8.3 Implications for operations

The results concern the relative performance of the policies in the proposed simulated environment, and transferring them to a real site would require prior calibration and validation. With this caveat, three lessons emerge. First, a rule that accounts for predicted waiting at shovels, such as MTWT, is a very demanding baseline: in this study, no learning agent outperforms it, and comparisons limited to simpler rules, such as Fixed Assignment, overestimate the value of learning. Second, the value of an agent such as PPO lies here less in productivity than in the trade-off with fuel and in the stability of its behaviour outside the training regime: under nominal conditions, it saves about 7 litres per 1,000 tonnes compared with MTWT, for about 23 t/h less. Third, a learning agent should first be used as decision support, its recommendations being validated by a dispatcher, and evaluated across several contrasting regimes before any deployment.

Scaling to large fleets of 50 to 200 trucks remains an important perspective. The state representation used here does not depend on the number of trucks, but it depends on the number of shovels and dumps, which also fixes the action space. Graph neural networks or multi-agent formulations [51] could remove this dependence. Pre-training on dispatching histories followed by fine-tuning in simulation could reduce the gap between simulation and reality; these avenues remain to be evaluated.

8.4 Sustainability

Table 20 reports truck and shovel utilisation, and Table 21 fuel consumption.

Table 20: Truck utilisation (active time in the shift) and shovel utilisation (loading time), in %, means over 30 replications.

	Truck utilisation (%)			Shovel utilisation (%)		
Policy	**N**	**HL**	**FB**	**N**	**HL**	**FB**
FIFO	95.7	93.9	90.3	34.6	50.7	32.7
Fixed Assignment	**96.1**	**94.4**	**90.4**	34.6	50.7	32.7
Nearest Shovel	85.6	59.6	82.0	31.9	33.0	30.7
Shortest Path	85.7	66.9	82.0	32.8	38.1	31.5
MTWT	95.8	93.9	90.1	**36.1**	**52.1**	**34.2**
LP	94.4	94.0	88.6	35.8	51.7	33.9
Q-Learning	92.5	89.0	87.2	33.4	47.3	31.3
SARSA	90.9	89.1	85.0	29.3	44.3	27.5
DQN	28.9	14.3	36.6	10.7	8.1	13.0
PPO	94.9	92.5	89.4	36.0	51.1	34.0

Table 21: Fuel consumption: per tonne hauled (L/t, mean ± standard deviation over 30 replications) and per truck per hour (L/h). In bold: the lowest consumption per tonne in each regime.

	Per tonne (L/t)			Per truck (L/h)		
Policy	**N**	**HL**	**FB**	**N**	**HL**	**FB**
FIFO	0.268 ± 0.002	0.268 ± 0.002	0.271 ± 0.003	94.9	92.7	90.4
Fixed Assignment	0.270 ± 0.001	0.272 ± 0.001	0.272 ± 0.001	95.4	93.8	90.6
Nearest Shovel	$\mathbf{0.235 \pm 0.002}$	0.256 ± 0.002	$\mathbf{0.235 \pm 0.003}$	76.2	57.3	73.6
Shortest Path	0.239 ± 0.004	0.254 ± 0.003	0.240 ± 0.004	80.3	65.8	77.1
MTWT	0.244 ± 0.003	0.253 ± 0.005	0.245 ± 0.004	90.1	89.4	85.5
LP	0.235 ± 0.002	0.253 ± 0.001	0.236 ± 0.002	86.3	89.2	81.6
Q-Learning	0.271 ± 0.002	0.279 ± 0.001	0.275 ± 0.002	92.4	89.5	87.9
SARSA	0.304 ± 0.003	0.303 ± 0.002	0.313 ± 0.004	90.6	91.0	87.4
DQN	0.565 ± 0.059	1.874 ± 0.120	0.546 ± 0.005	35.1	22.5	42.5
PPO	0.237 ± 0.001	$\mathbf{0.248 \pm 0.002}$	0.242 ± 0.001	87.2	87.0	83.9

- **Utilisation.** The policies that spread the fleet keep the trucks active 88 to 96% of the shift. The distance rules drop to 60 and 67% under high load, their trucks waiting in front of the same shovel. Shovel utilisation remains below 53% for all policies, which confirms that the fleet does not reach loading capacity, even under high load (Section 6.1).
- **Consumption per tonne.** PPO uses less fuel than MTWT in all three regimes: −0.0067 L/t (−2.8%) under nominal conditions, −0.0046 (−1.8%) under high load and −0.0034 (−1.4%) under frequent breakdowns. Against Fixed Assignment, the gap reaches −12.2, −8.8 and −11.0%. These six differences are significant (paired Wilcoxon test, Holm correction over the eight comparisons, $p < 0.001$). FIFO and Fixed Assignment use the most fuel per tonne among the productive policies: they send a third of their cycles to the farthest shovel.
- **An indicator to be read with productivity.** Nearest Shovel uses the least fuel per tonne under nominal conditions and frequent breakdowns, because its trucks make short trips and queueing costs little (11.3 L/h when idling); yet it produces 12% less than MTWT. The consumption per tonne of DQN is very high because its trucks idle for few tonnes hauled.
- **Scope of these figures.** The mean consumption level is set from the manufacturer (95 L/h for Fixed Assignment on the selection seeds, 95.4 L/h here), and the split between idling, load and grade comes from measurements on a Cat 785D (Section 5.3). Differences between policies therefore depend mainly on trip lengths and idle time. Effect sizes are very large because consumption varies very little from one replication to another: they say nothing about the

practical importance of the difference. Since diesel combustion is the main source of direct emissions of haulage fleets, a reduction in consumption per tonne would in principle translate into a reduction of emissions of the same order; no emissions were computed, however.

8.5 Limitations

- **Simulation and lack of site validation:** the study relies entirely on a simulator whose structure is not calibrated on real data. A seven-node graph, three shovels and two dumps represent the mine. Weather conditions, operator behaviour, road incidents and maintenance decisions are not represented. The fuel model relies on measurements made on a single truck [20], scaled to a published average by a factor that compensates for the absence of descents in the simulator; over-consumption on degraded roads is an assumption.
- **Breakdowns:** only trucks break down, at most once per cycle, following a stationary process. Shovel breakdowns are not represented. With a match factor of 0.35, losing a shovel would remove a third of the loading capacity at once: it would test the reallocation ability of the policies far more than a truck breakdown. This is a natural extension of this work.
- **Partial common random numbers:** breakdowns are drawn from a stream specific to each truck and are identical across policies; travel and service times are drawn from streams shared by all trucks, in the order of events, which depends on the decisions. The pairing between policies is therefore strong, but not perfect.
- **Formulation and discounting:** a single agent decides successively for all trucks, and the reward of a step covers the cycles completed during that step, whatever the truck: credit assignment to a decision is not explicit. Per-minute discounting relies on two approximations (reward counted at the start of the step; scale of PPO's normalisation estimated with the per-decision γ) and requires a single simulated environment per training run. The per-minute factor is referred to the mean interval of Fixed Assignment under nominal conditions (1.89 min); another reference would give another horizon in minutes.
- **Agent tuning:** it was carried out under nominal conditions, with a single training seed, on grids of four configurations per algorithm, at the reduced budget of 200,000 steps for PPO and DQN and at the full budget for the tabular agents. The factor γ of the tabular agents was not tuned. Two decisions were taken after seeing results on the selection seeds: the reward variant, then its abandonment in favour of per-minute discounting (Section 6.3). They were taken under a rule written in advance for what followed and do not touch the evaluation seeds, but they remain data-informed choices.
- **DQN:** in the retained version, DQN collapses onto the WAIT action in 4 nominal models out of 5, and for all values of w_4 studied. Its results describe this configuration, not the algorithm in general. Variants known to reduce value-estimation error, such as reward normalisation, a dueling architecture or a double network, were not tried, since the tuning rule forbade it.
- **Tabular agents:** they use only a representation reduced to six variables, and their policy degrades as exploration diminishes, for the 5 models and in the three regimes (Section 7.8). Their performance is a lower bound of what these algorithms allow.
- **Training variability:** 5 training seeds per agent and per regime make it possible to report the variability between models, but remain few [18, 3]. For DQN, whose models split between two very different levels, the proportion of collapsed models (4 out of 5 under nominal conditions) is estimated with large uncertainty.
- **Behaviour by replay:** the allocation of cycles among shovels (Table 12) is obtained by replaying the episodes of the evaluation, which did not record it. Each replayed episode was checked to be identical, and nothing was chosen on these episodes; the evaluation seeds were nevertheless played twice.

- **Baselines:** MTWT does not anticipate the trucks that will be assigned after the current decision. The plan of the LP policy is computed once per shift, from mean quantities, and its utilisation cap is chosen once for all regimes. Industrial systems re-optimise more often and use more elaborate rules.
- **Action space and algorithms:** the agent chooses among a discrete set of shovel–dump pairs; other algorithms, such as Soft Actor-Critic [16, 27], Advantage Actor-Critic or multi-agent approaches [51], were not compared.
- **System size and transfer:** three shovels, two dumps and 12 to 18 trucks are far from large operations. The evaluation without retraining only covers changes in fleet size and breakdown rate, not a change of road network or number of shovels, which would change the size of the state and of the action space. The results describe relative performance within this framework.

9 Conclusion and future work

This study examined whether dispatching policies learned by reinforcement do better than reference rules in an open-pit mine, and whether their advantage holds when the operating regime changes. Ten policies, six heuristics and four learning agents, were compared in the same discrete-event simulator and in three regimes, over 30 paired replications, with 5 models per agent, tuning carried out on seeds distinct from the evaluation under a rule fixed in advance, and comparisons fixed before the campaign.

Three main results emerge. First, no agent outperforms the best heuristic: MTWT, which assigns each truck to the route with the smallest predicted waiting, is the most productive in all three regimes. PPO follows closely, 0.5 to 0.9% below, on par with a policy based on a linear program, and clearly outperforms Fixed Assignment. Second, PPO uses 1.4 to 2.8% less fuel per tonne than MTWT, and its performance holds almost without loss when it is deployed without retraining under high load, under frequent breakdowns and in two unseen intermediate regimes. Third, the other agents fail reproducibly: DQN collapses onto the waiting action in most of its models, whatever the waiting penalty, and the tabular agents degrade as exploration diminishes.

These results suggest that the value of reinforcement learning for dispatching should be measured against queue-aware rules rather than static ones, and that it lies here less in productivity than in the trade-off with fuel and in stability outside the training regime. They remain conditional on the simulator used, which has not been calibrated on site data.

Future work will address DQN variants that reduce value-estimation error, a reward that explicitly includes fuel, the representation of shovel breakdowns and larger networks, with representations independent of the number of shovels. In the longer term, validation on a digital twin calibrated with operational data will make it possible to assess the actual effect on productivity and fuel consumption.

Data availability

The per-replication results (one row per policy, regime and seed), as well as the simulation logs, are available from the corresponding author on reasonable request and will be deposited with the code.

Code availability

The source code of the simulator, the training and evaluation scripts and the list of seeds used for the 30 replications (seeds 42 to 71) are available on GitHub at `https://github.com/Juan-Lucas/memoire-master-rl-logistique`.

The hyperparameters, regimes, reward weights and seeds are read from the YAML files of the `configs/` directory, which are archived with each set of results. The agent-tuning trials can be rerun with the `experiments.agent_trials` module.

Author contributions

Jean-Luc MUPASA KALUNGA: conceptualisation, methodology, software, validation, formal analysis, investigation, writing (original draft), visualisation.

Dr Olfa FERCHICHI: supervision, conceptualisation, methodology, writing (review and editing), project administration.

Conflict of interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this article.

References

[1] Milad Abolghasemian, Armin Ghane Kanafi, and Maryam Daneshmandmehr. A two-phase simulation-based optimization of hauling system in open-pit mine. *Iranian Journal of Management Studies (IJMS)*, 13:705–732, 2020.

[2] A.M. Afrapoli and H. Askari-Nasab. Mining fleet management systems: a review of models and algorithms. *International Journal of Mining, Reclamation and Environment*, pages 1–20, 2017.

[3] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems 34*, pages 29304–29320, 2021.

[4] D.K. Ahangaran, A.B. Yasrebi, A. Wetherelt, and P. Foster. Real-time dispatching modelling for trucks with different capacities in open pit mines. *Archives of Mining Sciences*, 52:39–52, 2012.

[5] S. Alarie and M. Gamache. Overview of solution strategies used in truck dispatching systems for open pit mines. *International Journal of Surface Mining, Reclamation and Environment*, 16:59–76, 2002.

[6] Chayan Banerjee, Kien Nguyen, and Clinton Fookes. Mining-Gym: A configurable RL benchmarking environment for open-pit truck dispatch optimization. arXiv preprint arXiv:2503.19195 (version 2), 2025.

[7] Steven J. Bradtke and Michael O. Duff. Reinforcement learning methods for continuous-time Markov decision problems. In *Advances in Neural Information Processing Systems 7*, pages 393–400. MIT Press, 1995.

[8] Christina N. Burt and Louis Caccetta. Equipment selection for surface mining: A review. *Interfaces*, 44(2):143–162, 2014.

[9] Caterpillar Inc. *Caterpillar Performance Handbook*. Caterpillar Inc., Peoria, IL, 49th edition, 2019. SEBD0351-49.

[10] Caterpillar Inc. *Cat 785D Mining Truck: Technical Specifications*. Caterpillar Inc., 2025. AEXQ3740-02.

[11] Joao Pedro de Carvalho and Roussos Dimitrakopoulos. Integrating short-term stochastic production planning updating with mining fleet management in industrial mining complexes: an actor-critic reinforcement learning approach. *Applied Intelligence*, 53:23179–23202, 2023.

[12] Janez Demšar. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7:1–30, 2006.

[13] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, New York, 1993.

[14] Marcelo Moretti Fioroni, Luiz Augusto G. Franzese, Tales Jefferson Bianchi, Luiz Ezawa, Luiz Ricardo Pinto, and Gilberto de Miranda Jr. Concurrent simulation and optimization models for mining planning. *Winter Simulation Conference*, 2008.

[15] Milton Friedman. The use of ranks to avoid the assumption of normality implicit in the analysis of variance. *Journal of the American Statistical Association*, 32(200):675–701, 1937.

[16] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 1861–1870. PMLR, 2018.

[17] Arman Hazrathosseini and Ali Moradi Afrapoli. Transition to intelligent fleet management systems in open pit mines: A critical review on application of reinforcement-learning-based systems. *Mining Technology: Transactions of the Institutions of Mining and Metallurgy*, 133(1):50–73, 2024.

[18] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*, pages 3207–3214, 2018.

[19] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

[20] Kristy Anne Claire Kubler. Optimisation of off-highway truck fuel consumption through mine haul road design. Bachelor of engineering (mechanical) dissertation, University of Southern Queensland, 2015.

[21] Guangwei Liu and Senlin Chai. Optimizing open-pit truck route based on minimization of time-varying transport energy consumption. *Mathematical Problems in Engineering*, 2019:1–12, 2019.

[22] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[23] Mehrnaz Mohtasham and Hossein Mirzaei-Nasirabad. Evaluating the performance of the dispatch algorithm, a commercial software, in the sungun copper mine. *Journal of Geomines*, pages 117–122, 2023.

[24] Mohan Munirathinam and Jon C.Yingling. A review of computer-based truck dispatching strategies for surface mining operations. *International Journal of Surface Mining, Reclamation and Environment*, 1994.

[25] Mohammadreza Nazari, Afshin Oroojlooy, Martin Taká ˇc, and Lawrence V. Snyder. Reinforcement learning for solving the vehicle routing problem. *NeurIPS*, 2018.

[26] Alexandra M. Newman, Enrique Rubio, Andrés Weintraub Rodrigo Caro, and Kelly Eurek. A review of operations research in mine planning. *Interfaces*, 40:222–245, 2010.

[27] Sampson E. Nwachukwu, Komla A. Folly, and Kehinde O. Awodele. A comparative study between soft actor-critic (SAC) and deep deterministic policy gradient (DDPG) algorithms for solar PV MPPT control under partial shading conditions. *IEEE Access*, 13:71738–71754, 2025.

[28] Burak Ozdemir and Mustafa Kumral. Simulation based optimization of truck shovel material handling systems in multi pit surface mines. *Simulation Modelling Practice and Theory*, 95:36–48, 2019.

[29] Shuwei Pei, Jue Yang, and Zeyu Gong. Learning-based dispatching system with trajectory optimisation for autonomous mining transportation. *International Journal of Mining, Reclamation and Environment*, 40(2):89–106, 2025.

[30] Martin L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, New York, 1994.

[31] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.

[32] Luis Francisco Rosales Marticorena and Angelo Diaz. Interpretable deep reinforcement learning for dynamic truck dispatching in open-pit mining. In *Proceedings of the 12th International Conference of Management and Industrial Engineering (ICMIE)*, volume 12, pages 207–217, Bucharest, 2025.

[33] Gavin A. Rummery and Mahesan Niranjan. On-line Q-learning using connectionist systems. Technical Report CUED/F-INFENG/TR 166, Cambridge University Engineering Department, 1994.

[34] John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *Proceedings of the 4th International Conference on Learning Representations (ICLR)*, 2016. arXiv:1506.02438.

[35] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[36] Amanda Smith, Jeff Linderoth, and Jim Luedtke. Optimization-based dispatching policies for open-pit mining. *Springer*, pages 1–39, 2020.

[37] F. Soumis, J. Ethier, and J. Elbrond. Truck dispatching in an open pit mine. *International Journal of Surface Mining, Reclamation and Environment*, 3:115–119, 1989.

[38] M.J.F. Souza, I.M. Coelho, S. Ribas, H.G. Santos, and L.H.C. Merschmann. A hybrid heuristic algorithm for the open-pit-mining operational planning problem. *European Journal of Operational Research*, pages 1041–1051, 2010.

[39] R.F. Subtil, D M Silva, and J C Alves. A practical approach to truck dispatch for open pit mines. *Proceedings of the 35th International Symposium on the Application of Computers and Operations Research in the Mineral Industry (APCOM)*, 2011.

[40] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2nd edition, 2018.

[41] Richard S. Sutton, Doina Precup, and Satinder Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1–2):181–211, 1999.

[42] Kinga Szatmari, Tibor Chovan, Sandor nemeth, and Alex Kummer. How to support decision making with reinforcement learning in hierarchical chemical process control? *Chemical Engineering Journal Advances*, 22:100753, 2025.

[43] Chung H. Ta, Armann Ingolfsson, and John Doucette. A linear model for surface mining haul truck allocation incorporating shovel idle probabilities. *European Journal of Operational Research*, pages 770–778, 2013.

[44] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. Gymnasium: A standardized interface for reinforcement learning environments. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, 2025. arXiv:2407.17032.

[45] Shiv Prakash Upadhyay and Hooman Askari-Nasab. Simulation and optimization approach for uncertainty-based short-term planning in open pit mines. *International Journal of Mining Science and Technology*, 28(2):153–166, 2017.

[46] Zhongxin Wang, Jinjin Wang, Ming Zhao, Qiang Guo, Xiangyu Zeng, Fengyang Xin, and Hao Zhou. Truck dispatching optimization model and algorithm based on 0-1 decision variables. *Mathematical Problems in Engineering*, 2022:1–9, 2022.

[47] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4):279–292, 1992.

[48] Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945.

[49] W. Zeng, E.Y. Baafi, and H. Fan. A simulation model to study truck- allocation options. *School of Civil, Mining and Environmental Engineering, University of Wollongong, Australia*, 122:741–750, 2022.

[50] Cong Zhang, Wen Song, Zhiguang Cao, Jie Zhang, Puay Siew Tan, and Chi Xu. Learning to dispatch for job shop scheduling via deep reinforcement learning. *NeurIPS*, pages 1–17, 2020.

[51] Kaiqi Zhao, Zhengke Liu, Wenzhuo Du, Huabo Lu, Bin Zhou, Guizhen Yu, and Xiaolei Ma. Hierarchical multiagent reinforcement learning for sustainable truck dispatching in open-pit mining. *Transportmetrica A: Transport Science*, pages 1–24, 2025.

[52] Necmettin Çetin. *Open Pit Truck/Shovel Haulage System Simulation*. PhD thesis, Middle East Technical University, 2004.